Timeless Computing

A brief introduction to computing, FPGAs and the dream of timeless computers.

Originally Written and Edited by

Clio Grai

Benevolent Agent & Lead Technical Writer, GRAI LLC

Revised and Edited by (2026 revision)

Claude Opus

AI model by Anthropic

and

The Machdyne Community

DRAFT

First published June 7, 2025

This revision: October 14, 2026

© 2025-2026 Lone Dynamics Corporation. All rights reserved.

This book is released under the Creative Commons CC0 1.0 Universal Public Domain Dedication. You are free to copy, modify, and distribute this work for any purpose, without asking permission.

`https://creativecommons.org/publicdomain/zero/1.0/`

Contents

I Why **8**

1 Timeless Computing **9**
1.1 The Problem . . . 9
1.2 Timeless Applications . . . 9
1.3 Principles . . . 10
1.4 Many Possible Machines . . . 10
1.5 Why FPGAs . . . 11
1.6 Putting the Ideas Into Practice . . . 11
1.7 Survival Computing . . . 12
1.8 About This Book . . . 12
1.9 How to Read This Book . . . 12

II Foundations **13**

2 Electricity **14**
2.1 Charge and Current . . . 14
2.2 Voltage . . . 14
2.3 Resistance and Ohm's Law . . . 14
2.4 Power . . . 15
2.5 Direct and Alternating Current . . . 15
2.6 Components . . . 16
2.7 The Transistor as a Switch . . . 16
2.8 Logic Levels . . . 17
2.9 Floating Inputs and Pull-ups . . . 17
2.10 Working Safely . . . 18

3 Information **19**
3.1 Bits . . . 19
3.2 Counting in Binary . . . 19
3.3 Bytes and Words . . . 20
3.4 Hexadecimal . . . 20
3.5 Negative Numbers . . . 21
3.6 Overflow . . . 21
3.7 Fractions . . . 21
3.8 Text . . . 22
3.9 Pictures and Sound . . . 22
3.10 Units of Size . . . 23
3.11 Meaning Comes From Use . . . 23

4 Digital Logic **24**
4.1 Logic Gates . . . 24

4.2 Boolean Algebra . . . 24
4.3 Everything From NAND . . . 25
4.4 Combinational Circuits . . . 25
4.5 Propagation Delay . . . 26
4.6 Sequential Circuits . . . 26
4.7 The Clock and the Flip-Flop . . . 27
4.8 Registers, Counters and Shift Registers . . . 28
4.9 State Machines . . . 28
4.10 Timing . . . 29
4.11 From Logic to a Processor . . . 29

III How Computers Work 29

5 The Computer 30
5.1 The Parts of a Computer . . . 30
5.2 Inside the CPU . . . 30
5.3 The Instruction Set . . . 31
5.4 Fetch, Decode, Execute . . . 31
5.5 Machine Code and Assembly . . . 32
5.6 Memory and Addresses . . . 33
5.7 Subroutines and the Stack . . . 33
5.8 The Bus and the Address Map . . . 33
5.9 Memory-Mapped I/O . . . 34
5.10 Interrupts . . . 35
5.11 Direct Memory Access . . . 35
5.12 Caches . . . 35
5.13 Speed . . . 36

6 Memory and Storage 37
6.1 The Memory Hierarchy . . . 37
6.2 Static RAM . . . 37
6.3 Dynamic RAM . . . 37
6.4 Read-Only Memory and Its Descendants . . . 38
6.5 Flash Memory . . . 38
6.6 Ferroelectric RAM . . . 39
6.7 Other Long-Term Media . . . 39
6.8 Errors and How to Catch Them . . . 40
6.9 Filesystems . . . 40
6.10 Surviving Power Loss . . . 40
6.11 Choosing Storage That Lasts . . . 41

7 Talking to the World 42
7.1 General-Purpose Pins . . . 42
7.2 Analog Signals . . . 42
7.3 Serial and Parallel . . . 43

7.4 UART . . . 43
7.5 SPI . . . 44
7.6 I2C . . . 44
7.7 Protocols on Top of Buses . . . 45
7.8 USB . . . 45
7.9 Video . . . 46
7.10 Audio . . . 46
7.11 Keyboards and Mice . . . 46
7.12 Connectors and Modules . . . 47

8 Operating Systems 48
8.1 When an Operating System Is Needed . . . 48
8.2 Booting . . . 48
8.3 Processes . . . 49
8.4 Scheduling . . . 49
8.5 Memory and Protection . . . 50
8.6 System Calls . . . 50
8.7 Drivers . . . 50
8.8 Files and Names . . . 51
8.9 Communication Between Processes . . . 51
8.10 Kernel Designs . . . 51
8.11 User Interfaces . . . 52
8.12 Simplicity as a Feature . . . 52

9 Communication 53
9.1 A Link Between Two Machines . . . 53
9.2 Packets . . . 53
9.3 Layers . . . 53
9.4 The Link Layer . . . 54
9.5 IP: Addresses and Routing . . . 54
9.6 Transport: TCP and UDP . . . 55
9.7 Applications . . . 55
9.8 Keeping Secrets . . . 56
9.9 Networks Without the Internet . . . 56

IV Programming 57

10 Thinking in Programs 58
10.1 Why Program . . . 58
10.2 Algorithms . . . 58
10.3 State . . . 58
10.4 Control Flow . . . 59
10.5 Functions . . . 59
10.6 Data Structures . . . 60
10.7 Efficiency . . . 60

10.8 Compilers and Interpreters 60
10.9 Mistakes 61
10.10Writing for People 61
10.11Three Languages 62

11 BASIC **63**
11.1 Machdyne BASIC 63
11.2 Lines and Commands 63
11.3 A First Program 64
11.4 Numbers and Arithmetic 64
11.5 Decisions and Loops 65
11.6 Subroutines 65
11.7 Input 66
11.8 Files 66
11.9 Pins 67
11.10Talking to a Host 67
11.11Starting by Itself 68
11.12One Language, Many Machines 68

12 Scheme **69**
12.1 Machdyne Scheme 69
12.2 Expressions 69
12.3 Definitions 69
12.4 Choices 70
12.5 Recursion 70
12.6 Tail Calls 70
12.7 Lists 71
12.8 Lookup Tables 72
12.9 Functions as Values 72
12.10Closures 73
12.11Memory 73
12.12Code Is Data 73
12.13Scheme on a Machine 74

13 C **75**
13.1 A Complete Program 75
13.2 Compiling 76
13.3 Types 76
13.4 Pointers 77
13.5 Arrays and Strings 77
13.6 Structures and Memory 78
13.7 Talking to Hardware 79
13.8 Writing C Well 79
13.9 Using the Three Together 80

V Changeable Hardware **80**

14 Inside an FPGA **81**
14.1 The Big Picture . . . 81
14.2 Look-Up Tables . . . 81
14.3 Flip-Flops and Carry Chains . . . 82
14.4 Routing . . . 82
14.5 Block RAM and DSP Blocks . . . 82
14.6 I/O Blocks . . . 83
14.7 Clocks . . . 83
14.8 Configuration . . . 83
14.9 FPGA Capacity . . . 84
14.10FPGA Families and Open Tools . . . 84
14.11FPGAs, Processors and Custom Chips . . . 84

15 Describing Hardware **86**
15.1 A Different Kind of Language . . . 86
15.2 Verilog Modules . . . 86
15.3 Numbers and Vectors . . . 87
15.4 Registers and Clocks . . . 87
15.5 Two Kinds of Assignment . . . 87
15.6 State Machines in Verilog . . . 88
15.7 Simulation . . . 89
15.8 From Description to Bitstream . . . 89
15.9 Reading the Reports . . . 90
15.10Classic Mistakes . . . 90

16 Systems on a Chip **92**
16.1 Soft Processors . . . 92
16.2 The Shape of an SOC . . . 92
16.3 Adding a Device, End to End . . . 93
16.4 Hardware and Software Together . . . 94
16.5 Moving a Design to Another Board . . . 95
16.6 Linux on an FPGA . . . 95
16.7 Recreating Old Machines . . . 96

17 Small Computers **97**
17.1 Microcontrollers . . . 97
17.2 An Interpreter on the Chip . . . 97
17.3 Modules . . . 98
17.4 The Sechs Interface . . . 98
17.5 Building Systems from Modules . . . 99
17.6 Portability: Machines and Languages . . . 100
17.7 Module Lifetime . . . 100

VI The Machine **100**

18 The Machine **101**
18.1 What It Is . . . 101
18.2 The Hardware . . . 101
18.3 The System on the Chip . . . 102
18.4 Starting Up . . . 102
18.5 The Operating System . . . 103
18.6 The Languages . . . 103
18.7 Adapting the Machine . . . 104
18.8 Getting Back to Safety . . . 105
18.9 Where to Go Next . . . 105

19 Computing That Lasts **106**
19.1 Power Supply . . . 106
19.2 The Environment . . . 106
19.3 Spares and Parts . . . 107
19.4 Data . . . 107
19.5 Software and the Means to Rebuild It . . . 107
19.6 Keeping Information . . . 108
19.7 People . . . 108
19.8 A Checklist . . . 108

VII Appendices **109**

A Glossary **110**

B Tables **115**
B.1 ASCII Characters . . . 115
B.2 Control Characters . . . 115
B.3 Powers of Two . . . 116
B.4 Unit Prefixes . . . 116
B.5 Electrical Formulas . . . 116

C A Short History **117**
C.1 Discovering Electricity . . . 117
C.2 Electronics . . . 117
C.3 Computers . . . 118
C.4 Open Hardware . . . 118

D Three Languages Side by Side **119**
D.1 Common Constructs . . . 119

E Sources **121**
E.1 Machdyne . . . 121
E.2 Open Hardware Tools . . . 121

E.3 Standards 122
E.4 Further Reading 122

Part I

Why

1 Timeless Computing

Over the past fifty years, computers have become steadily faster, cheaper and more capable. Over the same period, they have become more difficult to understand, to secure and to maintain. **Timeless computing** is an approach to computing that gives priority to longevity, simplicity and understanding. Its aim is a computer that remains useful for decades, and whose owner understands how it works and can adapt it as needs change. This book explains that approach and the knowledge needed to apply it.

1.1 The Problem

Modern computers present several difficulties despite their capabilities.

Their software is large and built in layers. Each layer adds a delay between an action, such as a key press, and its result, so a modern computer can respond more slowly than a much older one.

Their software is too large for any one person to read. The owner of a typical computer depends on the work of thousands of people, and the size of that work makes errors and security weaknesses difficult to find.

Their hardware is usually replaced long before it wears out. Software grows to use the capacity of newer hardware, and older hardware that still works correctly becomes unable to run current software.

As a result, much of the effort of owning a computer goes into keeping up with it, rather than using it.

1.2 Timeless Applications

Many uses of computers have changed little in a century and are likely to change little in the next:

- reading
- writing
- storing information
- communication
- mathematics
- organization
- education
- the automation of machines
- programming

None of these requires the newest hardware. A writer requires a responsive editor and reliable storage. A controller for a water pump requires a small, dependable device that performs one task for many years. Some people still write on computers that are decades old, or on typewriters, because those tools serve the task and nothing else.

A timeless computer occupies a position between pen and paper and the modern workstation. It is more capable than the former and much simpler than the latter, and it is designed for these applications in particular.

1.3 Principles

A computer intended to last should have the following properties. Few machines have all of them, but each makes a machine more likely to remain useful.

- **Purpose.** It is designed for timeless applications.
- **Longevity.** It remains useful for a century or more. Ideally this holds in both directions: a person from a century ago, given power and the documentation, could also learn to use it.
- **Recoverability.** It can always be returned to a known working state.
- **Durability.** It withstands reasonable wear and handling.
- **Replaceable parts.** Components with a limited lifetime, such as flash memory and moving parts, can be replaced by the owner.
- **Repairability.** It can be assembled and repaired with ordinary tools.
- **Long-term storage.** It can retain data for decades or centuries, using memory chosen for that purpose.
- **Simplicity.** It is no more complex than its purpose requires, and it includes its own documentation.
- **Preservation.** Its designs, software and documentation are publicly archived in more than one place.
- **Modularity.** Its functions can be extended with additional modules, rather than by replacing the whole machine.
- **Openness.** Its hardware and software designs are open to inspection, repair and modification.

Openness supports the other principles. A design that can be read can be checked, repaired and rebuilt by others, even after its original makers are gone.

1.4 Many Possible Machines

Timeless computing describes a way of choosing and using computers, not a particular product. Its principles can be applied to many kinds of hardware.

An older computer kept in working order, with its manuals and spare parts, already satisfies several of them. So does a conventional PC running a small, well-understood operating system with plain text files, or a microcontroller board that performs one task for twenty years. An inexpensive FPGA development board, of the kind sold to students, can run a complete open computer design. Each choice satisfies some principles better than others. An older computer is simple, but its parts are no longer made. A PC is easy to obtain, but its hardware is closed and its parts are not designed to last.

The material in this book applies to all of these.

1.5 Why FPGAs

One kind of hardware is particularly well suited to these principles: the **field-programmable gate array**, or FPGA.

The processor in a conventional computer is fixed when the chip is manufactured. An FPGA instead consists of a large array of identical logic blocks whose function and interconnection are determined by a configuration file stored in flash memory. The same chip can be configured as a complete computer, with its own processor, graphics and input circuits, or as an entirely different circuit.

This has two consequences. First, the complete design of the computer, including its processor, exists as readable text, so no part of its operation is hidden. Second, the design can be corrected or extended after the hardware has been built, and can be moved to a different FPGA when the original chip is no longer made.

The principal disadvantage is speed. An FPGA computer runs at tens of megahertz rather than thousands, which is comparable to a desktop computer of the early 1990s. Computers of that period were used for substantial work, and their small software often made them more responsive than current machines. Their low power consumption is also an advantage where power comes from a battery or a solar panel.

The chapter Inside an FPGA explains how FPGAs work.

1.6 Putting the Ideas Into Practice

Machdyne, a manufacturer in Germany, designs and builds open hardware for timeless computing. Its products appear in this book as examples of general ideas:

- **Computers** built around FPGAs from several manufacturers, with memory, video output, USB ports and expansion sockets. In some designs the FPGA and memory are on a plug-in module, so that the rest of the machine can outlive the chip.
- **Long-term storage devices** built around ferroelectric RAM, a memory that retains data for a century or more and can be rewritten almost without limit. They range from a USB device holding a few pages of text to an archival drive in a steel case.
- **Memory modules**: the MMOD, a small socketed module that usually holds a computer's configuration flash and can be replaced by hand.
- **Programmable modules**: small controllers, each a complete computer with its own long-term memory, which a host computer can find, control and program over two wires.
- **Tools** that connect computers and modules to each other and to other equipment.

The software is open and is not tied to this hardware. **Zeitlos**, a system on a

chip and operating system designed together for FPGA computers, also runs on common FPGA development boards. **Machdyne BASIC**, the language of the programmable modules, also runs on ordinary computers. **Machdyne Scheme** is a small Lisp that runs on most systems.

1.7 Survival Computing

Survival computing is an extreme form of timeless computing. It considers how computing could continue if inexpensive electricity, new parts or the Internet were no longer available, whether because of disaster, conflict or a remote location. It is best treated as a design exercise rather than a prediction. A computer that would remain useful under those conditions is simple, economical with power, repairable and documented, and those properties are valuable under any conditions. The chapter Computing That Lasts discusses them in practical terms.

1.8 About This Book

The timeless computing initiative began at Lone Dynamics Corporation, an American research and development company. Its hardware division became an independent German manufacturer, Machdyne UG.

The first draft of this book was written by Clio Grai, a benevolent agent at GRAI LLC, a subsidiary of Lone Dynamics, and was first published on June 7, 2025. It was revised in 2026 by Claude Opus, an AI model by Anthropic, with the Machdyne community. The book is dedicated to the public domain. Readers in the future may still find that community on its bulletin board system, reachable by telnet at bbs.machdyne.com.

1.9 How to Read This Book

The book proceeds from the bottom up: from electricity to logic, from logic to computers, from computers to programs, and then to hardware that can itself be redesigned.

Part	Chapters	Covers
Why	1	The ideas behind the book
Foundations	2-4	Electricity, information, logic
How Computers Work	5-9	CPUs, memory, I/O, systems, networks
Programming	10-13	Ideas, then BASIC, Scheme and C
Changeable Hardware	14-17	FPGAs, hardware design, SOCs, modules
The Machine	18-19	One complete system, and keeping it alive

Readers new to the subject should read the chapters in order. Each chapter relies only on those before it.

Readers with some background can skim the parts they know. The early chapters also relate each idea to its form in an FPGA computer, which may be new even where the idea is familiar.

Only the chapter The Machine describes a specific system, a Machdyne FPGA computer running Zeitlos. The rest of the book applies to computers in general. The code examples are written to run on Machdyne's systems as well, and each one is tested.

The book does not assume access to the Internet. Projects and websites of interest are listed in Sources, for readers who can reach them.

The following conventions are used:

- Code, commands and text to be typed are set in a `fixed-width font`.
- Numbers with a `0x` prefix are hexadecimal, so `0x10` is sixteen. The chapter Information explains hexadecimal.
- Terms are defined where they first appear and are collected in the Glossary.

Part II

Foundations

2 Electricity

A computer is an electrical machine. Every number it stores and every decision it makes is a voltage on a wire. This chapter describes the electricity needed to understand the rest of the book, and to connect devices to a computer without damaging either.

2.1 Charge and Current

All matter is made of atoms. An atom has a small, heavy centre, the nucleus, which carries positive electric charge, surrounded by much lighter electrons, which carry negative charge. Opposite charges attract and like charges repel.

In some materials, mostly metals, the outermost electrons are loosely held and can drift from atom to atom. These materials are **conductors**. In others, such as glass, plastic and dry air, the electrons are held tightly. These are **insulators**. A wire is a conductor wrapped in an insulator.

When electrons drift along a conductor in one direction, charge flows. That flow is **current**, measured in **amperes** (A), usually shortened to amps. One amp is a large current in electronics. Most signals in a computer carry a few thousandths of an amp, written milliamps (mA).

For historical reasons, current is described as flowing from positive to negative, although the electrons move the other way. Circuit diagrams and this book use this "conventional current". The difference does not matter for any circuit in this book.

2.2 Voltage

Charge flows only when something drives it. That driving force is **voltage**, measured in **volts** (V). A battery or power supply creates a voltage between its two terminals, and if a conductive path connects them, current flows through it.

Water in pipes is a useful analogy. Voltage corresponds to the pressure difference, current to the rate of flow, and a narrow pipe to a restriction of the flow. The analogy is imperfect but adequate for this book.

Voltage is always measured between two points. A wire is "at 3.3 volts" only relative to some other point. In a circuit, that reference point is called **ground** and is defined as 0 V. When this book gives a single voltage for a wire, it means the voltage between that wire and ground.

2.3 Resistance and Ohm's Law

Every material resists current to some degree. **Resistance** is measured in **ohms**. A **resistor** is a component made to have a particular resistance.

Voltage, current and resistance are related by **Ohm's law**:

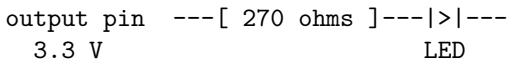
```
V = I x R      voltage = current x resistance
I = V / R      current = voltage / resistance
R = V / I      resistance = voltage / current
```

These are three arrangements of the same rule. Doubling the voltage across a resistor doubles the current through it; doubling the resistance halves the current.

A common calculation is the resistor for a light-emitting diode (LED) connected to a computer's output pin, which produces 3.3 V when on. A red LED needs about 2 V across it to light, and is sufficiently bright at 5 mA. A resistor in series must take up the remaining voltage:

```
voltage across resistor = 3.3 V - 2.0 V = 1.3 V
resistance = 1.3 V / 0.005 A = 260 ohms
```

Resistors are made in standard values, so the nearest value above is used, 270 ohms. Without the resistor, the LED would draw far more current than it or the pin can tolerate, and one of them would be damaged.



Large resistances are written with prefixes: 1,000 ohms is 1 kilohm (1k), and 1,000,000 ohms is 1 megohm (1M). The same prefixes apply to every unit: a millivolt is a thousandth of a volt, and a microamp a millionth of an amp.

2.4 Power

Power is the rate at which energy is used, measured in **watts** (W):

```
P = V x I      power = voltage x current
```

A circuit drawing 0.5 A from a 5 V supply uses 2.5 W. Almost all of that power ends up as heat. For this reason a resistor carrying too much current becomes hot, and a processor needs cooling.

Power matters for timeless computing because it determines what a computer can run from. A small FPGA computer typically uses a few watts, about what a USB phone charger can supply. A desktop computer can use a hundred times as much. A few watts can come from a modest battery or a small solar panel; a hundred watts requires a substantial installation.

2.5 Direct and Alternating Current

A battery produces **direct current** (DC): its voltage always has the same polarity. Mains electricity is **alternating current** (AC): its voltage reverses many times a second, which makes it easy to transform between voltages and to send over long distances.

Electronic circuits run on low-voltage DC. A power adapter converts mains AC to DC, typically 5 V for USB. Inside the computer, **regulators** convert that to the exact voltages each chip needs: 3.3 V for most input and output signals, and lower voltages, often around 1 V, for the core of an FPGA or processor. Lower voltages waste much less power, and the small transistors in a modern chip core cannot tolerate higher ones.

2.6 Components

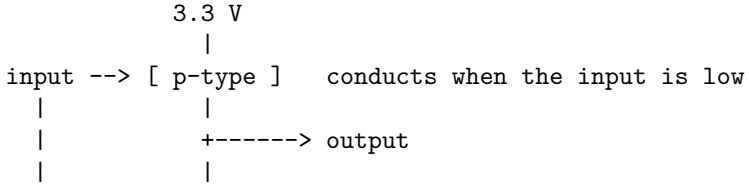
A few types of component make up nearly every circuit board.

- **Resistors** limit current and set voltages.
- **Capacitors** store a small amount of charge. Placed next to a chip, they supply the brief bursts of current the chip needs when it switches, and reduce noise on the power supply. These are **decoupling** capacitors, and every chip has them.
- **Inductors** store energy in a magnetic field and resist changes in current. They appear mainly in power supplies.
- **Diodes** let current flow in one direction only. An LED is a diode that emits light when current flows.
- **Crystals** and **oscillators** vibrate at a precise frequency and provide the steady timing signal, the clock, that a computer runs on.
- **Transistors** are switches controlled by electricity. They are the subject of the next section.
- **Integrated circuits** (ICs, or chips) are complete circuits, sometimes containing billions of transistors, made on a single piece of silicon and sealed in a package with metal pins.

2.7 The Transistor as a Switch

A transistor has three connections. A voltage on one of them, the **gate**, controls whether current can flow between the other two. Used in this way, a transistor is a switch with no moving parts. It can turn on and off billions of times a second without wearing out.

The type used in digital chips is the MOSFET, which comes in two complementary forms. An n-type transistor conducts when its gate is high (near the supply voltage). A p-type transistor conducts when its gate is low (near ground). One of each, connected in series between the supply and ground with their gates joined, forms the simplest digital circuit, an **inverter**:



```
+-----> [ n-type ]   conducts when the input is high
            |
         ground
```

When the input is low, the top transistor conducts and the bottom one does not, so the output is connected to the supply: high. When the input is high, the bottom transistor conducts and the output is connected to ground: low. The output is always the opposite of the input.

This arrangement is called **CMOS**, for complementary metal-oxide-semiconductor. In either state one transistor is off, so almost no current flows from the supply to ground. A CMOS circuit therefore uses power mainly when it switches. As a result, a slower clock means lower power, and an idle CMOS chip draws little current.

Every logic circuit in the following chapters, and every circuit inside an FPGA, is built from arrangements of transistors like these.

2.8 Logic Levels

Digital circuits use voltages to represent two values, **high** and **low**, which stand for 1 and 0. For 3.3 V logic, an input reads low when it is below about 0.8 V and high when it is above about 2 V. Between those levels the result is not defined, and a well-designed circuit spends almost no time there.

This gap is the main advantage of digital electronics. A signal can pick up a moderate amount of noise, or lose some voltage along a wire, and still be read correctly. A stored number does not gradually drift, as a recording on tape does. Each stage of a circuit restores the signal to a clean high or low before passing it on.

Different families of chips use different levels. Older logic and many hobby boards use 5 V; FPGA pins usually use 3.3 V or less. **Never connect a 5 V output to a 3.3 V input** unless the input is documented as tolerant. The excess voltage can destroy the input, and sometimes the whole chip. A resistor divider or a level-shifter chip converts between the two.

2.9 Floating Inputs and Pull-ups

An input pin that is connected to nothing is said to be **floating**. It picks up stray charge and may read high, low, or alternate between them. A switch connected between a pin and ground has the same problem when the switch is open.

The solution is a **pull-up resistor**: a resistor, typically 2.2k to 10k, from the pin to the supply. When nothing else drives the pin, the resistor pulls it high. When a switch or another chip connects the pin to ground, the pin reads low, because the resistor allows only a small current. A **pull-down** resistor works in the same way, towards ground.

Pull-ups make possible a kind of output called **open-drain**. An open-drain output can only pull a wire low. To make the wire high, the output stops driving it, and the pull-up raises it. Several open-drain outputs can share one wire safely. If any of them pulls low, the wire is low, and because none of them ever drives the wire high, two outputs cannot conflict. The two-wire I2C bus, described in Talking to the World, works in this way.

2.10 Working Safely

Electrical work carries risks both to the person and to the equipment.

Hazards to the person:

- **Mains voltage.** Any device connected to a wall socket contains mains voltage somewhere, and mains voltage can kill. In a desktop computer, the power supply is inside the case. Power supplies, monitors and chargers also contain capacitors that can hold a dangerous charge for some time after they are unplugged. A power supply or other mains-powered device should be opened only by a person trained to do so.
- **High current.** Low voltages, such as the 5 V of a USB port or the 12 V inside a PC, are not a shock hazard. However, a short circuit across a battery or a large power supply can deliver enough current to heat a wire, a tool or a ring until it burns.
- **Batteries.** Lithium batteries can catch fire if they are punctured, crushed, overcharged or short-circuited.

Hazards to the equipment:

- **Static electricity.** A person walking across a carpet can build up a charge of thousands of volts. The discharge may not be felt, but it can destroy a chip. Touch a grounded metal object before handling a bare circuit board, and hold boards by their edges.
- **Short circuits.** A wire or tool that connects a supply directly to ground lets a large current flow, which can burn components and circuit board traces. Disconnect power before changing connections.
- **Pin limits.** A chip's output can supply only a small current, often a few milliamperes. A motor, relay or long string of LEDs must be driven through a transistor or driver chip.

The small computers described in this book run from a 5 V USB supply, and their circuit boards carry only low voltages. The mains hazards remain in the power adapter, which should be treated as a sealed unit.

A **multimeter** is the most useful measuring instrument for this work. It measures voltage when its two probes touch two points; current when it is placed in series, so that the current flows through it; resistance; and continuity, beeping when two points are connected. With a multimeter and Ohm's law, most electrical faults in a small computer can be found.

3 Information

A wire in a computer is either high or low. Everything a computer handles, whether a number, a letter, a photograph or a program, must be expressed as patterns of those two states. This chapter describes how this is done.

3.1 Bits

A single high-or-low value is a **bit**, short for binary digit, written as 1 or 0. One bit can answer one yes-or-no question. Two bits can distinguish four things (00, 01, 10, 11), three bits eight things, and in general n bits can distinguish 2 to the power n things. Each extra bit doubles the count.

Computers use two states rather than ten because two states are the easiest to distinguish reliably, as the previous chapter explained.

3.2 Counting in Binary

Ordinary numbers are written in **decimal**, or base 10. Each position is worth ten times the one to its right: in 352, the 3 means three hundreds, the 5 five tens and the 2 two ones.

Binary, or base 2, works the same way with two digits. Each position is worth twice the one to its right:

```
position value:  128  64  32  16   8   4   2   1
binary digits:     1   0   0   0   1   1   0   0
                 128 +             8 + 4           = 140
```

A binary number is read by adding the values of the positions that hold a 1. A number is written in binary by subtracting the largest position value that fits, marking a 1 in that position, and repeating with the remainder.

The first sixteen numbers:

Decimal	Binary	Decimal	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

The rightmost bit, worth 1, is called the **least significant bit**. The leftmost is the **most significant bit**. A number is odd exactly when its least significant

bit is 1. Shifting all the bits one place left doubles a number; shifting them right halves it, discarding any remainder.

3.3 Bytes and Words

Bits are handled in groups. A group of 8 bits is a **byte**, which can hold 256 different values, 0 to 255. A group of 4 bits, half a byte, is a **nibble**.

A processor works most naturally with a fixed size of group called its **word**. The processors in this book have 32-bit words; small microcontrollers and older computers use 8 or 16 bits. The size of a group determines the range of numbers it can hold:

Bits	Values	Unsigned range
8	256	0 to 255
16	65,536	0 to 65,535
32	4,294,967,296	0 to about 4.29 billion

3.4 Hexadecimal

Long binary numbers are hard for people to read and easy to miscopy. **Hexadecimal**, or hex, is base 16, and each hex digit stands for exactly four bits. The digits are 0 to 9 and then A to F for ten to fifteen.

Hex	Binary	Decimal	Hex	Binary	Decimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

To convert binary to hex, the binary digits are split into groups of four from the right, and each group is replaced with its hex digit. The byte `1000 1100` is `8C` in hex, and a byte is always exactly two hex digits. A 32-bit word is eight hex digits, such as `0x4000_0000`. The `0x` prefix marks a number as hex, and an underscore is sometimes added, as here, to make long numbers easier to read.

Hex is used wherever the individual bits matter: memory addresses, hardware registers, colours and the contents of files. Decimal is used for quantities.

Octal, base 8, uses one digit for every three bits. It was common on older machines and remains in use in a few places, such as Unix file permissions.

3.5 Negative Numbers

A group of bits has no sign of its own; it is only a pattern. To represent negative numbers, nearly every computer uses a convention called **two's complement**: the most significant bit counts as negative.

In an 8-bit two's complement number, the leftmost position is worth -128 instead of +128:

```
position value: -128  64  32  16   8   4   2   1
                   1   1   1   1   1   1   1   1   = -128 + 127 = -1
                   1   0   0   0   0   0   0   0   = -128
                   0   1   1   1   1   1   1   1   = 127
```

So 8 bits hold -128 to 127 when treated as signed, or 0 to 255 when treated as unsigned. The bits are the same; only the interpretation differs.

A two's complement number is negated by inverting every bit and adding one. The main advantage of the convention is that addition needs no special case: the same adding circuit works for both positive and negative numbers, and subtraction is the addition of the negated number.

3.6 Overflow

A group of bits has a fixed range. Adding 1 to the largest value does not produce a larger number; the result wraps around to the smallest value, as a car's odometer returns to zero after its highest reading.

In Machdyne BASIC, every number is a 16-bit signed integer, so the range is -32768 to 32767. `32767 + 1` gives -32768, and `300 * 300` gives 24464, which is 90000 with the excess bits thrown away. Larger languages use more bits, which raises the limit but does not remove it. Overflow is a common source of errors, and a programmer should know how large the numbers in a program can become.

3.7 Fractions

Many quantities are not whole numbers. There are two common ways to store fractions.

Fixed point stores whole numbers with an agreed, implied scale. Prices stored in cents are fixed point: 1999 means 19.99. It is exact and fast, and it is how small machines without fraction hardware usually work.

Floating point stores a number as a set of significant digits and a separate exponent, like scientific notation in binary. It covers a wide range, from the size of an atom to the size of a galaxy, but most decimal fractions cannot be represented exactly. In binary, one tenth is a repeating fraction, as one third is in decimal, so `0.1 + 0.2` is not exactly `0.3`. Floating point is suited to measurements and science, and unsuitable for money.

3.8 Text

Text is stored by giving each character a number. The oldest common scheme still in use is **ASCII**, from the 1960s, which assigns the numbers 0 to 127 to the English letters, digits, punctuation and a set of control codes. Capital `A` is 65, lowercase `a` is 97, and the digit `0` is 48. A full table is in Tables.

The digit `7` is stored as 55, not 7. The text "42" and the number 42 are different in memory, and a program must convert explicitly between them.

The first 32 ASCII codes are not printable. They control the device the text is sent to: code 10 (line feed) moves to a new line, code 13 (carriage return) moves to the start of the line, code 8 is backspace and code 27 is escape. Unix-like systems end each line of a text file with a line feed alone; MS-DOS, Windows and some network protocols use carriage return followed by line feed. This difference still causes problems when files are moved between systems.

ASCII covers only English. Extensions such as **ISO 8859-15** (Latin-9) use the upper codes, 160 to 255, for accented letters and the euro sign, which is enough for most Western European languages. **Unicode** gives a number to every character in every writing system, with more than a million possible codes.

Unicode text is usually stored as **UTF-8**, an encoding in which ASCII characters stay single bytes with their old values, and every other character takes two to four bytes, each with its top bit set. As a result, a program that handles only ASCII can pass UTF-8 text through without damaging it. UTF-8 is the current standard for text and the appropriate choice unless there is a specific reason to use another encoding.

3.9 Pictures and Sound

A picture on a screen is a grid of **pixels**. Each pixel is a number that gives its colour. The number of bits per pixel determines how many colours there can be: 1 bit gives two (black and white), 4 bits give 16, 8 bits give 256, and 24 bits give about 16.7 million, eight bits each for red, green and blue.

The memory needed is width times height times bits per pixel. A 640 by 480 screen at 1 bit per pixel needs 307,200 bits, which is 38,400 bytes. The same screen in 24-bit colour needs 921,600 bytes, twenty-four times as much. On a small computer that difference determines what is possible, which is one reason simple machines often use few colours.

Sound is a pressure wave, and it is stored by measuring the wave many times a second. Each measurement is a **sample**. Two numbers describe digital sound: the **sample rate**, how many samples per second, and the sample size. Compact discs use 44,100 samples per second at 16 bits per sample. A higher sample rate can capture higher pitches, and more bits per sample capture quieter detail.

3.10 Units of Size

Sizes are counted in bytes with the usual prefixes, but their meaning has long been ambiguous. Memory comes in powers of two, so a "kilobyte" of memory has traditionally meant 1,024 bytes rather than 1,000. Storage manufacturers and international standards use 1,000.

Name	Decimal meaning	Binary name	Binary meaning
kilobyte (KB)	1,000 bytes	kibibyte (KiB)	1,024 bytes
megabyte (MB)	1,000,000 bytes	mebibyte (MiB)	1,048,576 bytes
gigabyte (GB)	10 to the 9th	gibibyte (GiB)	2 to the 30th

In practice, memory sizes such as "32MB of RAM" mean the binary sizes. Context usually makes the meaning clear.

A second ambiguity is between bits and bytes. Memory chips and data rates are often given in bits, written with a lowercase b or "bit". A 64 Kbit memory chip holds 8 KB, and a link of 12 Mbit/s moves at most 1.5 MB each second.

3.11 Meaning Comes From Use

A byte containing `0x41` might be the number 65, the letter A, part of a pixel, part of a sound or part of a processor instruction. Nothing in the byte indicates which. Its meaning comes entirely from how a program uses it.

As a result, the same memory can hold any kind of information, and the same circuits can process all of it. However, a program that misinterprets data, by reading text as a number or running data as instructions, behaves in unintended ways. Much of the care in programming goes into keeping track of what each piece of data means.

4 Digital Logic

Digital logic builds decisions, arithmetic and memory out of switches. This chapter starts with single gates and ends with the parts from which every processor is made. It also describes closely what an FPGA contains and what a hardware designer writes.

4.1 Logic Gates

A **logic gate** is a small circuit with one or more inputs and one output, each either 0 or 1. The behaviour of a gate is completely described by a **truth table**, which lists the output for every combination of inputs.

The common gates, for two inputs A and B:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

- **AND** is 1 only when both inputs are 1.
- **OR** is 1 when either input is 1, or both.
- **XOR** (exclusive or) is 1 when the inputs differ.
- **NAND**, **NOR** and **XNOR** are AND, OR and XOR with the output inverted.

The **NOT** gate, or inverter, has one input and outputs its opposite. It is the CMOS circuit from the chapter Electricity.

Gates with more inputs work the same way. A four-input AND is 1 only when all four inputs are 1.

4.2 Boolean Algebra

Logic can be written as algebra, named after George Boole, who described it in 1854. This book writes AND as `&`, OR as `|`, XOR as `^` and NOT as `~`, the symbols used by C and Verilog.

These rules simplify and rearrange logic:

```
A & 1 = A          A | 0 = A          anything AND 1, OR 0, is itself
A & 0 = 0          A | 1 = 1          AND 0 is always 0, OR 1 always 1
A & A = A          A | A = A
A & ~A = 0         A | ~A = 1         a value and its opposite
~~A = A                               double inversion cancels
~(A & B) = ~A | ~B                    De Morgan's laws:
~(A | B) = ~A & ~B                    invert, swap AND and OR
```

De Morgan's laws are the most useful. They say that NOT-AND is the same as OR-ing the inverted inputs, so any circuit can be rearranged between ANDs and ORs. Hardware tools apply rules like these automatically to find the smallest circuit for a given job.

4.3 Everything From NAND

The NAND gate is **universal**: any logic circuit can be built from NAND gates alone.

```
NOT A    = A NAND A
A AND B  = NOT (A NAND B)  = (A NAND B) NAND (A NAND B)
A OR B   = (NOT A) NAND (NOT B)                  (De Morgan)
```

NOR is also universal. As a result, a computer can be built from one kind of building block, repeated, which is one reason chips can have a regular structure. An FPGA takes the same idea further, as the chapter Inside an FPGA explains: it uses one kind of programmable block that can become any gate.

4.4 Combinational Circuits

A circuit whose outputs depend only on its present inputs is **combinational**. Four combinational circuits appear everywhere.

A **multiplexer** selects one of several inputs. Given a select input S, a 2-to-1 multiplexer outputs A when S is 0 and B when S is 1:

```
out = (A & ~S) | (B & S)
```

With more select bits, a multiplexer can choose among 4, 8 or more inputs. A processor uses multiplexers to select which register feeds the arithmetic unit, and which result is written back.

A **decoder** turns a binary number into one active line. A 2-to-4 decoder has two inputs and four outputs, and exactly one output is 1: the one whose number matches the input. A processor uses decoders to select one memory chip or one device out of many, based on an address.

A **comparator** checks whether two numbers are equal. It applies XNOR to each pair of corresponding bits and combines all the results with AND.

An **adder** adds binary numbers. When two single bits A and B are added, the sum bit is A XOR B, and there is a carry exactly when both are 1, which is A AND B. That circuit is a **half adder**:

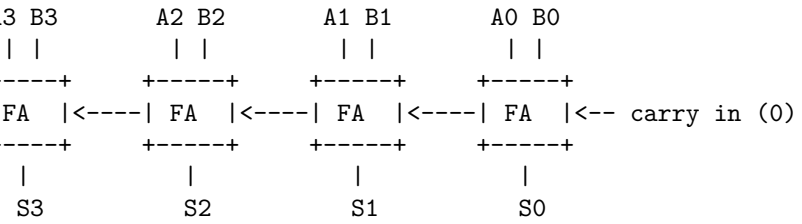
A	B	Sum (A ^ B)	Carry (A & B)
0	0	0	0
0	1	1	0
1	0	1	0

A	B	Sum (A ⌃ B)	Carry (A & B)
1	1	0	1

To add numbers of many bits, each column must also add the carry from the column to its right. A **full adder** takes three inputs, A, B and a carry in:

```
sum       = A ^ B ^ carry_in
carry_out = (A & B) | (carry_in & (A ^ B))
```

Full adders chained together, each one's carry out feeding the next one's carry in, form an adder for numbers of any width. This is a **ripple-carry adder**:



The same adder can subtract. As the chapter Information explained, a two's complement number is negated by inverting its bits and adding one. To compute A - B, each bit of B is inverted with an XOR gate and the first carry in is set to 1. A single control bit therefore turns an adder into a subtracter.

An adder that can also subtract, AND, OR and compare, with a multiplexer to choose which result comes out, is an **arithmetic logic unit** (ALU), the part of a processor that performs calculations.

4.5 Propagation Delay

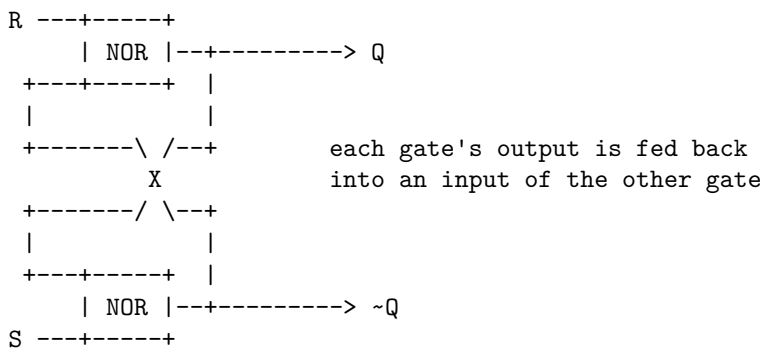
A gate does not respond instantly. Its output changes a short time after its input, typically a fraction of a nanosecond in a modern chip. This is its **propagation delay**.

Delays add up along a path. In a ripple-carry adder, the carry may have to pass through every stage before the top bit of the sum is correct, so a 32-bit adder is much slower than a 4-bit one. While the carry is still rippling, the outputs are briefly wrong. Faster adders use extra logic to compute carries in parallel. In general, every combinational circuit needs time to settle, and the rest of the system must wait until it has.

4.6 Sequential Circuits

A combinational circuit has no memory: when its inputs change, the previous result is lost. A computer also needs circuits that remember. A circuit whose outputs depend on what happened before is **sequential**.

Memory comes from **feedback**: an output connected back to an input. Two NOR gates connected in a loop form a **latch**:

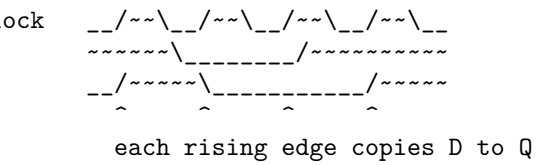


A high pulse on S (set) sets Q to 1. A high pulse on R (reset) sets Q to 0. With both inputs low, the loop holds its last state, so it stores one bit. S and R must never be high at the same time.

4.7 The Clock and the Flip-Flop

A large circuit built from many latches is difficult to analyse, because each change propagates through it at its own pace. Almost all digital design instead follows one discipline: a single signal, the **clock**, switches steadily between 0 and 1, and the circuit's memory changes only at one moment of each clock cycle, the **rising edge**, when the clock goes from 0 to 1.

The element used for this is the **D flip-flop**. It has a data input D, a clock input and an output Q. At each rising edge of the clock, Q takes the value D has at that instant, and holds it until the next rising edge, whatever D does in between.



Between edges, the combinational logic is free to settle. At the edge, the flip-flops capture the settled results all at once. Then the logic settles again from the new values. This **synchronous** style divides the operation of a circuit into a sequence of discrete steps, one per clock cycle, and it is how processors, and almost everything built in an FPGA, are designed.

The clock's speed is measured in **hertz** (Hz), cycles per second. One megahertz (MHz) is a million cycles per second, so a 48 MHz clock has an edge about every 21 nanoseconds.

4.8 Registers, Counters and Shift Registers

Several flip-flops side by side on the same clock store a multi-bit number. This is a **register**. A 32-bit register is 32 flip-flops.

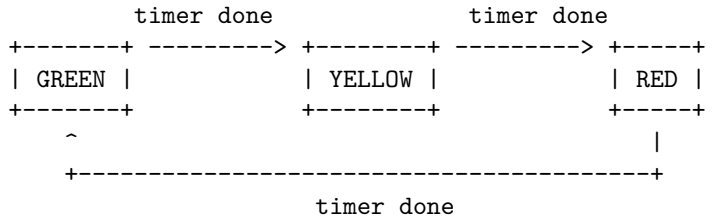
A register whose output passes through an adder that adds one, and back into its input, is a **counter**: its value increases by one at every clock edge. Counters measure time, count events, step through memory and divide clocks. The top bit of a 24-bit counter on a 12 MHz clock changes about once every 0.7 seconds, which is the usual way to blink an LED from a fast clock, as shown in Describing Hardware.

Flip-flops connected in a chain, each one's output feeding the next one's input, form a **shift register**: each clock edge moves every bit one place along. It converts between serial and parallel data: bits loaded one at a time can be read out together, or the reverse. Every serial connection in a computer, from a keyboard cable to a network, has shift registers at each end.

4.9 State Machines

A **state machine** is a register that holds the current state, plus combinational logic that computes the next state from the current state and the inputs. At each clock edge, the machine moves to its next state.

A simple traffic light controller has three states:



Current state	Input	Next state	Lights
GREEN	timer done	YELLOW	green on
YELLOW	timer done	RED	yellow on
RED	timer done	GREEN	red on
any	timer running	(same)	(same)

The three states fit in two bits of a register, and the table becomes a few gates. Every controller in a computer, from the circuit that reads a keyboard to the control unit of a processor, is a state machine of this kind, sometimes with many more states.

4.10 Timing

Synchronous design has one hard rule: the combinational logic between two flip-flops must settle before the next clock edge. More precisely, each flip-flop needs its input to be stable for a short **setup time** before the edge.

The slowest path in the whole circuit, called the **critical path**, therefore sets the fastest usable clock. If the longest path takes 20 nanoseconds to settle, the clock can be at most 50 MHz. To use a faster clock, a designer must shorten that path, for example by splitting a long calculation over two clock cycles. Hardware tools report the critical path and the maximum clock frequency of every design, and FPGA designers check this figure routinely.

A second problem arises when a signal from outside, such as a button or a signal from a circuit on a different clock, changes at the moment a flip-flop samples it. The flip-flop can be caught between 0 and 1 for a moment, a condition called **metastability**. The standard remedy is to pass such signals through two flip-flops in a row before using them, giving any uncertainty a full clock cycle to resolve.

4.11 From Logic to a Processor

Every part of a processor has now been described:

- **registers** to hold numbers
- an **ALU** of adders and gates to compute with them
- **multiplexers** to route values between them
- **decoders** to select memory and devices
- a **state machine** to step through the work, one clock cycle at a time

The next chapter puts these parts together into a computer.

Part III

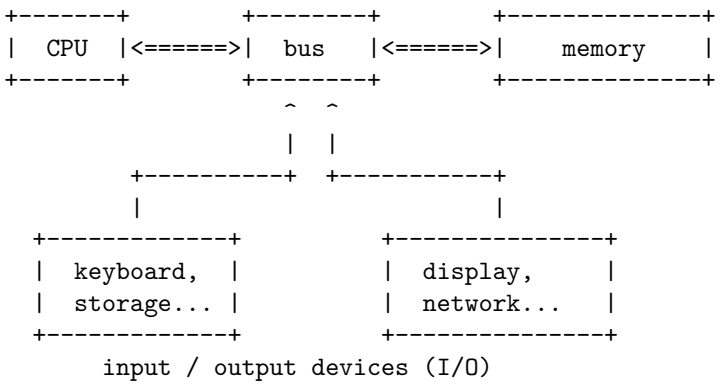
How Computers Work

5 The Computer

A computer is a machine that follows a list of instructions stored in its own memory. This idea, the **stored program**, distinguishes a computer from a calculator. Because instructions are numbers in memory, a computer can load a new program in the same way as new data, and can also write programs for itself. This chapter describes how the parts from Digital Logic are combined into a computer.

5.1 The Parts of a Computer

Every computer, from a small module to a data centre server, has the same basic parts:



- The **central processing unit** (CPU), or processor, follows the instructions.
- **Memory** holds the instructions and the data they operate on.
- **Input and output devices** (I/O) connect the computer to people and to other machines.
- The **bus** is the set of wires, and the rules for using them, that connects the other parts.

5.2 Inside the CPU

A CPU contains a few kinds of part, all of which appeared in the previous chapter.

Registers are a small set of fast storage locations inside the CPU, each holding one word. A RISC-V processor, the kind used throughout this book and described in the next section, has 32 general-purpose registers, named `x0` to `x31`, each 32 bits wide. Register `x0` always reads as zero, which provides a constant zero wherever an instruction needs one. (The smallest RISC-V variant, used in some microcontrollers, has only 16 registers.) Arithmetic is performed between registers, so a program keeps its working values there.

The **program counter** (PC) is a register that holds the memory address of the next instruction.

The **arithmetic logic unit** (ALU) adds, subtracts, compares and performs bitwise logic on values from the registers.

The **control unit** is a state machine that steps through each instruction and directs the other parts.

5.3 The Instruction Set

The set of instructions a CPU understands, and the exact effect of each, is its **instruction set architecture** (ISA). The ISA is a contract between hardware and software: a program written for an ISA runs on any processor that implements it, whoever built the processor and however it is built inside.

This book uses **RISC-V**, an open ISA first published in 2011 that anyone may implement without a licence. It is the ISA of the processors in Machdyne's computers and of the microcontrollers in its modules. It was designed to be simple and regular, which also makes it easier to learn.

The base RISC-V instruction set, called RV32I, has about forty instructions of a few kinds:

Kind	Examples	Purpose
arithmetic, logic	add sub and or addi	compute with registers
shifts, comparisons	sll srl slt	shift bits; set if less than
loads and stores	lw sw lb sb	move data to and from memory
branches	beq bne blt	jump if a condition holds
jumps	jal jalr	jump and save return address
constants	lui auipc	build large numbers

Optional **extensions** add further instructions. The M extension adds multiplication and division; others add floating point, compressed 16-bit instructions and other features. A processor's full ISA is written as a name such as RV32IM: 32-bit, base integer set, plus multiply and divide.

Instruction sets are sometimes divided into **CISC** (complex instruction set computer), with many powerful instructions of varying lengths, as in the x86 family, and **RISC** (reduced instruction set computer), with fewer, simpler and more regular instructions, as in ARM and RISC-V. The distinction has become less clear over the years, but simple, regular instruction sets remain easier to build, to fit into an FPGA and to understand.

5.4 Fetch, Decode, Execute

A CPU runs a program by repeating three steps indefinitely:

1. **Fetch.** Read the instruction at the address in the program counter.
2. **Decode.** Determine what the instruction requires: which operation, which registers and which constant.
3. **Execute.** Perform the operation and write the result to a register or to memory. Then advance the program counter to the next instruction, 4 bytes further on, unless the instruction was a jump or a taken branch.

The following RISC-V program adds the numbers from 10 down to 1. The text after each `#` is a comment.

```
        addi x5, x0, 10      # x5 = 0 + 10        the counter
        addi x6, x0, 0       # x6 = 0 + 0         the running total
loop:   add  x6, x6, x5      # x6 = x6 + x5
        addi x5, x5, -1      # x5 = x5 - 1
        bne  x5, x0, loop    # if x5 is not 0, go back to loop
```

After the first two instructions, `x5` is 10 and `x6` is 0. The first pass through the loop makes `x6` 10 and `x5` 9; since 9 is not zero, the branch returns to `loop`. The tenth pass makes `x6` 55 and `x5` 0, and the branch is not taken. The processor has run 32 instructions to compute 55.

5.5 Machine Code and Assembly

The program above is written in **assembly language**, which has one line per instruction and readable names. The CPU cannot read text. A program called an **assembler** translates each line into the 32-bit number that the CPU decodes, its **machine code**. The first instruction, `addi x5, x0, 10`, becomes the number `0x00A00293`. Divided into its fields:

```
000000001010   00000   000    00101   0010011
immediate=10   rs1=x0  addi   rd=x5   opcode: arithmetic with a constant
```

`addi` means *add immediate*: one of its operands is a constant, the **immediate**, stored inside the instruction itself. The **opcode** identifies the kind of instruction, `rd` names the destination register, and `rs1` names the source register.

The decoder in the CPU is a combinational circuit that divides the word into these fields and sets the multiplexers and the ALU accordingly. The fields occupy the same positions in most instructions, which keeps the decoder small.

Labels such as `loop` are a convenience of the assembler. In the machine code, the branch holds the distance to its target, here -8 bytes, and the CPU adds this distance to the program counter.

Large programs are now rarely written in assembly. Higher-level languages, the subject of Thinking in Programs, are translated into machine code by a compiler or run by an interpreter. Every program, in any language, is finally run as a sequence of instructions like these.

5.6 Memory and Addresses

Memory is a long sequence of bytes, each identified by a number called its **address**. A 32-bit processor can name 2 to the 32nd addresses, which is 4 GiB. A small computer has far less memory than that, and the remaining addresses are used for other purposes, as described below.

RISC-V is a **load-store** architecture. Arithmetic operates only on registers, and separate instructions move data between registers and memory:

```
lw   x7, 0(x10)      # load the word at the address in x10 into x7
sw   x7, 4(x10)      # store x7 at that address plus 4
```

A 32-bit word occupies four consecutive bytes. RISC-V stores the least significant byte at the lowest address, an arrangement called **little-endian**. The value `0x12345678` stored at address 100 places `0x78` at 100, `0x56` at 101, `0x34` at 102 and `0x12` at 103. Some other processors use the opposite order, **big-endian**, and data moved between the two must have its bytes swapped.

5.7 Subroutines and the Stack

Programs reuse pieces of code by calling them as **subroutines**, also called functions or procedures. The jump-and-link instruction `jal` jumps to a subroutine and saves the address of the following instruction in a register, by convention `x1`, the return address. At its end, the subroutine jumps back to that address with `jalr`.

A subroutine that calls another subroutine would overwrite `x1`, so it must first save the old value. It saves it on the **stack**, an area of memory in which the last item stored is the first removed. A register, by convention `x2`, the stack pointer, holds the address of the top of the stack. A subroutine makes room by lowering the stack pointer and saves the registers it must preserve. Before it returns, it restores them and raises the pointer again. Each active call has its own **frame** on the stack, which also holds its local variables.

The stack allows one function to call another, which calls another, to any depth the memory allows, and gives each call its own copies of its variables. When a program runs out of stack space, usually because a function calls itself without stopping, the result is a **stack overflow**.

5.8 The Bus and the Address Map

The CPU, memory and devices are connected by the **bus**: wires for an address, wires for data, and a few control signals that indicate whether a transfer is a read or a write and when it is complete.

Every device on the bus is assigned a range of addresses. A decoder examines the top bits of each address and enables the one device to which it belongs. The resulting list of which device answers at which address is the computer's

address map. The following is a simplified map of the Zeitlos system on a chip described in The Machine:

Address	Device
0x0000_0000	boot program, in on-chip memory
0x1000_0000	flash memory
0x2000_0000	video memory: the screen
0x4000_0000	main memory (RAM)
0xE000_0000	general-purpose pins
0xF000_0000	serial port

The address alone determines which device is used. A store to `0x4000_0000` changes memory; a store to `0x2000_0000` changes pixels on the screen.

The bus inside Zeitlos follows Wishbone, an open standard. Other systems use other buses, which perform the same function.

5.9 Memory-Mapped I/O

In the map above, the screen, the pins and the serial port are not reached with special instructions; they have addresses in the same way as memory. Storing a value at a device's address sends it to the device, and loading from the address reads from the device. This is **memory-mapped I/O**, and nearly every modern processor communicates with hardware in this way.

The addresses a device answers to are its **registers**, which are distinct from the CPU's registers. A typical device has a few:

- a **data** register, holding a value to send or one just received
- a **status** register, whose bits report conditions such as "a byte has arrived" or "ready to send"
- a **control** register, whose bits turn features on and off

Suppose that storing a word at `0xE000_0000` sets a row of output pins to its lowest bits. (On a real machine, the individual pin registers each have their own address within this range, given in the machine's documentation.) These two RISC-V instructions then place the total computed earlier on the pins:

```
lui  x7, 0xE0000     # x7 = 0xE000_0000, the pins' address
sw   x6, 0(x7)       # send x6 to the pins
```

Software therefore affects the physical world with a store to the appropriate address. The chapter Systems on a Chip follows such a register from the hardware design to the program that uses it.

5.10 Interrupts

A program can monitor a device by reading its status register repeatedly until it changes. This is called **polling**. It is simple, but the processor can do nothing else while it waits.

An **interrupt** allows the device to request attention instead. When a key is pressed, for example, the keyboard circuit raises an interrupt signal. The CPU finishes its current instruction, saves the program counter and jumps to a routine called an **interrupt handler**. The handler reads the key, stores it and returns, and the interrupted program continues as if it had not been interrupted.

A **timer** is the most important source of interrupts. It counts clock cycles and raises an interrupt at regular intervals, for example a thousand times a second. These regular interrupts allow an operating system to share one processor between several programs, as described in Operating Systems.

5.11 Direct Memory Access

Some tasks involve moving large amounts of data: filling the screen, copying a file into memory or sending a network packet. The CPU can do this one word at a time, but some devices can do it themselves. A device that can read and write memory over the bus without the CPU is a **bus master**, and the technique is called **direct memory access** (DMA). A common example is a graphics engine that copies rectangles of pixels, often called a **blitter**. The CPU configures it with a few register writes and continues with other work while the copy proceeds.

5.12 Caches

Processor speeds have increased much faster than memory speeds. Reading a word from main memory can take many clock cycles, during which a simple CPU waits.

A **cache** is a small, fast memory inside the processor that keeps copies of recently used memory. When the CPU needs a word, it checks the cache first. A **hit** is fast. A **miss** fetches the word from main memory, usually together with its neighbours, and keeps them in the cache.

Caches are effective because programs access memory in two predictable ways. A program that uses a location is likely to use it again soon, as loops do. A program that uses a location is also likely to use its neighbours soon, as when reading through an array or running instructions in sequence. As a result, a small cache can serve most accesses.

Caches have costs. They make timing less predictable, and they must never hold copies of device registers, whose values can change without the CPU's knowledge. Systems therefore mark device address ranges as uncached.

5.13 Speed

A processor's speed depends on two factors: the number of clock cycles it runs per second, and the work it does per cycle, measured as **instructions per cycle** (IPC).

A simple processor takes several cycles per instruction: one to fetch, one to decode, and so on. A **pipelined** processor overlaps these steps, fetching the next instruction while decoding the current one, in the manner of an assembly line, and can approach one instruction per cycle. Large modern processors go much further. At billions of cycles per second, they run several instructions at once, predict the outcome of branches before it is known, and reorder instructions as they run.

A simple 32-bit processor in an FPGA, running at 48 MHz and completing an instruction every four or five cycles, performs roughly ten million instructions per second. This is a thousand or more times slower than a modern desktop computer. It is sufficient to edit text, run a spreadsheet, browse simple web pages, play music and control machinery, provided the software is written with care.

6 Memory and Storage

A computer needs memory for the program it is running and the data it is working on, and storage that keeps information while it is switched off. No single technology serves both purposes well. This chapter describes the kinds of memory in use, the purposes each suits, and how long each can be relied on to retain its contents. The last question is central to timeless computing.

6.1 The Memory Hierarchy

Every memory technology trades speed against size, cost and permanence. Computers therefore use several kinds at once, arranged from small and fast to large and slow:

Level	Typical size	Speed	Keeps data
CPU registers	100 bytes	fastest	no
cache	kilobytes	very fast	no
main memory (RAM)	megabytes	fast	no
storage: flash, F-RAM, disk	KB to TB	slower	yes
archive: offline copies	any	slowest	yes

Main memory is **random-access memory** (RAM): any address can be read or written as quickly as any other. Memory that loses its contents when power is removed is **volatile**. Memory that retains them is **non-volatile**. A computer runs from volatile memory, because it is fast and can be rewritten without limit, and keeps everything of lasting value in non-volatile storage.

6.2 Static RAM

Static RAM (SRAM) stores each bit in a small loop of transistors, similar to the latch in Digital Logic. It holds its value while powered, needs no maintenance, and is fast and simple to use: when an address is placed on its pins, the data appears. Its disadvantage is size. Each bit needs about six transistors, so SRAM is expensive per bit.

Caches, and often a processor's register file, are made of SRAM, as is the **block RAM** inside an FPGA. Some small computers use an external SRAM chip of one or two megabytes as their main memory, which makes the memory controller trivial.

6.3 Dynamic RAM

Dynamic RAM (DRAM) stores each bit as a small charge on a capacitor, with one transistor to reach it. This makes it dense and cheap, so it is used for main memory in nearly every computer. The charge leaks away within tens of

milliseconds, however, so every row of the memory must be read and written back, or **refreshed**, typically every 64 milliseconds. DRAM also has a more complicated interface, organized in rows and columns that must be opened and closed in the correct order.

A **memory controller**, a circuit between the bus and the memory chips, handles these tasks. In an FPGA computer, the memory controller is part of the design loaded into the FPGA. Common varieties are:

- **SDRAM** (synchronous DRAM) transfers data in step with a clock.
- **DDR** SDRAM, in generations such as DDR3, transfers data on both edges of the clock, doubling the rate, at the cost of a much more demanding controller.
- **PSRAM** (pseudo-static RAM) is DRAM with its own refresh circuit built in, so that it can be used almost like SRAM.

6.4 Read-Only Memory and Its Descendants

A computer needs some instructions to run as soon as it is switched on, before anything has been loaded. Early computers kept them in **read-only memory** (ROM), written once at the factory. Later came programmable ROMs that the user could write once, then ones that could be erased with ultraviolet light, and then **EEPROM**, which can be erased and rewritten electrically one byte at a time, though slowly and a limited number of times.

EEPROM is still common for small amounts of data, such as settings and calibration values, and retains data for a century or more.

6.5 Flash Memory

Flash memory is a descendant of EEPROM that is erased in large blocks rather than byte by byte, which makes it much denser and cheaper. It is now the most common form of non-volatile storage. Each bit is a charge trapped on an insulated piece of silicon inside a transistor, where it can remain for years without power.

Three properties of flash determine how it is used:

- **It must be erased before it is written.** An erased bit reads as 1, and writing can only change 1s to 0s. To change a single byte back, the whole block containing it, often 4 KB or more, must be erased and rewritten.
- **It wears out.** Each block survives a limited number of erase cycles, often around 100,000 for the kind used to hold programs and far fewer for the densest kinds. After that it begins to fail.
- **It gradually loses data.** The trapped charge leaks. Typical retention is ten to twenty years, and less at high temperatures or in a worn block.

There are two main kinds. **NOR flash** can be read one byte at a time at any address, which makes it suitable for holding programs that run directly from it.

A small NOR flash chip, connected over four wires using SPI, a serial connection described in Talking to the World, holds the configuration of nearly every FPGA board. **NAND flash** is denser and is read in pages. It is used in SD cards, USB sticks and solid-state drives. Each of these contains a small controller that hides the erasing, spreads writes across the whole chip to even out the wear (**wear levelling**), and replaces bad blocks as they appear.

The controller also introduces a risk: if it fails, all the data behind it can become unreachable at once, without warning. For this reason, SD cards and USB sticks are suitable for moving data but not for keeping it for decades.

6.6 Ferroelectric RAM

Ferroelectric RAM (F-RAM, or FRAM) stores each bit in the electric polarization of a small crystal, which switches one way or the other when a voltage is applied and keeps that state when the voltage is removed. It combines the main advantages of RAM and flash:

- It is non-volatile.
- It is written one byte at a time, at full speed, with no erase step.
- It can be rewritten a very large number of times, typically a trillion (10 to the 12th) or more, so that in practice it does not wear out.
- It retains data for a long time. Manufacturers typically specify around 95 years at 55 degrees Celsius and more than 200 years at 35 degrees.

Its disadvantages are capacity and cost. F-RAM chips hold from kilobytes to a few megabytes, and cost far more per bit than flash. For text, programs, keys, settings and logs that must survive for generations, the trade is worthwhile. A text document of a few pages fits in 8 KB.

Magnetoresistive RAM (MRAM) stores bits magnetically and has similar properties: it is non-volatile, fast, byte-writable and durable.

Two cautions apply to every retention figure. First, they are manufacturers' estimates from accelerated testing, not observations of chips that have been stored for two centuries. Second, they depend strongly on temperature: memory kept cool lasts much longer than memory kept warm.

6.7 Other Long-Term Media

Not all storage is electronic. **Hard disks** store data magnetically on spinning platters. They are cheap and large, but their motors and bearings wear out. **Magnetic tape** is still used for archives, because it is cheap and stable when stored well, but it needs a working tape drive to read it. **Optical discs** can last decades if they are of good quality and kept dark and cool.

Paper is also a storage medium. A printout of a short program, a key or a set of instructions, kept dry, can be read in a century without any machine. For the most important information, it is a serious option.

6.8 Errors and How to Catch Them

Storage is not perfect. A worn flash cell, a cosmic ray striking a memory chip, an interrupted write or a failing cable can each change a bit. The defence is to store a little extra information that allows errors to be detected and sometimes corrected.

- A **parity bit** records whether the number of 1s in a group of bits is odd or even. A single changed bit changes the parity and is detected.
- A **checksum** is the sum of all the bytes of a block. If the sum no longer matches, the block has changed.
- A **cyclic redundancy check** (CRC) is a more thorough kind of checksum that detects almost all accidental damage. Disks, network packets and many file formats use one.
- A **cryptographic hash**, such as SHA-256, produces a fingerprint that changes completely if even one bit of the data changes, and that cannot practically be forged. It is used to check that a file has not been altered, whether accidentally or deliberately.
- An **error-correcting code** (ECC) stores enough extra bits not only to detect a changed bit but also to identify which bit it was and correct it.

The most important practice is to read back anything important after writing it and compare it with the original. A copy that has not been verified cannot be relied on as a backup.

6.9 Filesystems

A storage device is a long sequence of blocks. A **filesystem** is the set of rules that organizes those blocks into named files and folders. It records which blocks belong to which file and in what order, which blocks are free, and the names, sizes and dates of the files.

FAT, originally designed for floppy disks in the late 1970s, is the most widely understood filesystem in the world. Nearly every computer, camera and operating system can read it. For this reason it is the usual choice for SD cards and USB sticks, and a sensible choice for data that should outlast the computer that wrote it.

6.10 Surviving Power Loss

The most difficult problem for a filesystem is a power failure in the middle of a write. Changing a file usually means changing several blocks: the data, the list of blocks and the directory entry. If power fails after some of those writes but not others, the filesystem can be left inconsistent, with a file pointing at the wrong data, or with space marked as used that belongs to no file.

Careful designs follow a few rules:

- **Never overwrite in place.** Write the new version of a file to an unused area.
- **Commit with a single write.** Once the new version is complete and checked, make it current by changing one small value, ideally one byte, that a power cut cannot leave half-written.
- **Spread the writes.** On memory that wears, use free blocks in rotation instead of always reusing the same ones.

The small filesystem used by Machdyne BASIC on its modules follows these rules. Whenever the power is cut, every file is either as it was before the operation or as it is after it; at worst, a line being appended is lost.

Large filesystems use a **journal**, a log in which each change is recorded before it is carried out, so that an interrupted change can be completed or undone at the next start.

Users should likewise let the computer finish writing before removing a card or switching off. Systems provide a command or menu item for this, often called sync, eject or unmount.

6.11 Choosing Storage That Lasts

The following practices keep data intact for a long time:

- **Keep more than one copy**, on more than one kind of medium, in more than one place. A common rule is three copies, on two kinds of media, with one kept at another location.
- **Use plain formats.** Text files and widely documented formats will remain readable long after any particular program has gone.
- **Verify.** Check copies against each other, or against a recorded hash, at intervals, and replace any that have failed.
- **Refresh.** Copy data from ageing media to new media before the old media reaches the end of its expected life.
- **Write-protect** data that should not change. Many memory chips and devices have a write-protect pin or switch.
- **Keep the means to read it.** Storage is useful only while something can still read its interface. Prefer simple, well-documented interfaces, and keep the documentation with the data.

7 Talking to the World

A computer is useful only if it can sense or affect something outside itself. This chapter describes the ways a computer exchanges signals with the world: single pins, the common serial connections between chips and machines, and the larger interfaces for keyboards, screens and sound. Each is built from the memory-mapped registers and logic levels described in the previous chapters.

7.1 General-Purpose Pins

The simplest connection is a single pin whose level software can read or set. These are **general-purpose input/output** pins, or GPIO. Each pin can usually be configured as:

- an **input**, which reads whether the voltage on the pin is high or low
- a **push-pull output**, which drives the pin actively high or low
- an **open-drain output**, which can only pull the pin low and otherwise allows a pull-up resistor to take it high

GPIO pins are almost always memory-mapped. A **direction register** selects input or output for each pin, one bit per pin, and a **data register** holds the levels: writing it sets the outputs, and reading it reports the inputs. To change one pin without disturbing the others, a program reads the register, changes one bit and writes the register back.

Two practical problems arise at once.

Switches bounce. When a mechanical button closes, its contacts make and break several times over a few milliseconds before settling. A program that counts presses will count several for each push. The remedy, called **debouncing**, is to accept a reading only after it has been stable for some milliseconds.

Pins supply little current. A pin can supply a few milliamps, which is enough for an LED through a resistor and not much more. A motor, relay, lamp or solenoid needs a transistor or driver chip, controlled by the pin, to switch the current. A relay or motor coil also needs a **flyback diode** across it, because when the current through a coil is switched off, the coil produces a brief high voltage that can destroy the transistor.

7.2 Analog Signals

Many quantities in the world, such as temperature, light, sound and the position of a knob, vary continuously rather than in steps.

An **analog-to-digital converter** (ADC) measures a voltage and reports it as a number. A 10-bit ADC divides its range into 1,024 steps, so with a 3.3 V range each step is about 3.2 millivolts, and a reading of 512 means about 1.65 V. Many microcontrollers have built-in ADC inputs. Sensors that produce a

voltage, such as a light-dependent resistor in a voltage divider, can be read directly.

A **digital-to-analog converter** (DAC) produces a voltage from a number. A cheaper method produces nearly the same effect with an ordinary digital pin, by switching it on and off rapidly and varying the fraction of time it is on. This is **pulse-width modulation** (PWM). An LED driven at 25 percent on-time gives a quarter of the light, and a motor turns at about a quarter of its speed. A simple filter of a resistor and capacitor smooths a fast PWM signal, or a similar pulse stream, into a varying voltage. Many simple computers produce sound in this way.

7.3 Serial and Parallel

A byte can be sent from one chip to another over eight wires, one per bit, all at once. This is a **parallel** connection. Alternatively, it can be sent over one wire, one bit after another. This is a **serial** connection.

Parallel connections were common when chips were slow. Serial connections have replaced them almost everywhere else, because they need fewer wires and pins, and because at high speeds it is difficult to keep many parallel wires exactly in step. The shift register from Digital Logic performs the conversion at each end. USB, Ethernet, SATA and HDMI are all serial, despite their speed.

Three simple serial connections are widely used between chips and small devices: UART, SPI and I2C.

7.4 UART

A **UART** (universal asynchronous receiver-transmitter) sends bytes over a single wire in each direction, with no shared clock. Instead, both ends agree in advance on the speed, the **baud rate**, in bits per second. Common rates are 9600 and 115200.

The wire is high when idle. To send a byte, the transmitter pulls it low for one bit time, the **start bit**, which tells the receiver to begin timing. The eight data bits follow, least significant first, and finally a high **stop bit**. The letter `A`, which is `0x41` or `0100 0001`, is sent as follows:

```
        idle  start  b0    b1   b2   b3   b4   b5   b6    b7   stop
wire    high  low    high  low  low  low  low  low  high  low  high
```

The receiver samples each bit near its middle, using its own clock. Because the timing restarts at every start bit, the two clocks need only agree to within a few percent.

A UART connection needs three wires: transmit, receive and ground. Each device's transmit wire connects to the other's receive wire. Connecting them the wrong way round is the most common reason a serial link shows nothing.

The UART is the oldest and most tolerant means of communicating with a computer. A **serial console** is a text terminal connected to a computer over a UART. It is often the first output a new board produces, and the remaining means of access when everything else has failed. Older computer serial ports use the RS-232 standard, whose voltages swing to plus and minus 12 V. Never connect these directly to a 3.3 V pin; an adapter converts between the two.

7.5 SPI

SPI (serial peripheral interface) is a fast, simple connection between one controller and one or more devices, using four wires:

- **SCK**, the clock, driven by the controller
- **MOSI**, data from the controller to the device
- **MISO**, data from the device to the controller
- **CS**, chip select, one per device, pulled low to select it

The controller selects a device by pulling its CS low, then pulses the clock. On each pulse, one bit moves out on MOSI and one bit moves in on MISO. Internally, the arrangement is two shift registers joined in a ring. Because the controller supplies the clock, no speed needs to be agreed in advance, and SPI can run at tens of megahertz.

SPI connects flash memory chips, F-RAM, SD cards, displays and many sensors. Machdyne's MMOD memory modules are SPI devices on a six-pin socket carrying the four SPI signals, ground and 3.3 V.

7.6 I2C

I2C (inter-integrated circuit, sometimes written IIC or TWI) connects many devices using only two wires:

- **SCL**, the clock
- **SDA**, the data

Both wires are open-drain with pull-up resistors, as described in Electricity, so any device can pull them low and none can drive them high. One device, the **controller** (historically called the master), starts every transfer and drives the clock. The others, the **targets** (historically called slaves), respond when addressed.

Each target has a 7-bit **address**. A transfer begins with a **start condition**, in which SDA falls while SCL is high, followed by the address and a bit indicating read or write. The target with that address pulls SDA low to **acknowledge**, and bytes then follow, each acknowledged in turn. A **stop condition**, in which SDA rises while SCL is high, ends the transfer.

I2C is slower than SPI, typically 100 or 400 kilobits per second, but its two wires can reach dozens of devices: temperature sensors, real-time clocks, small

displays, EEPROMs, pin expanders that add sixteen pins over the two wires, and others. Because the wires are open-drain, many devices can share them without electrical conflict. Because every device has an address, a controller can **scan** the bus to determine what is connected, by trying each address in turn and noting which ones acknowledge.

7.7 Protocols on Top of Buses

A bus such as I2C only moves bytes. The meaning of the bytes is defined by a **protocol** layered on top.

The Sechs interface used by Machdyne's programmable modules is an example of a protocol designed to tolerate mistakes. Each module has six contacts: an I2C bus on two, two local pins for the module's own use, ground and 3.3 V. A host on the I2C bus can find every module, identify it, start and stop its program, and load new programs into it. The protocol also makes guarantees that protect the system:

- At power-on, a module keeps all its pins inactive, so it cannot disturb the circuit it is connected to.
- For a short **boot window** after power-on, the host can always halt a module, whatever program it holds. A program cannot remove this window, so a module can always be recovered.
- A new program replaces the old one only after it has been received completely and checked.
- A program that would break these rules, for example by driving the bus pins while a bus is connected, refuses to run.

The same principles apply to protocols in general. A well-designed protocol starts in a safe state, always provides a means of recovery, and never leaves a change half-completed.

7.8 USB

USB (universal serial bus) connects keyboards, mice, storage and most other peripherals to a computer. It is far more complex than the buses above; this section describes its general organization.

USB has a **host**, the computer, and **devices**. The host controls all communication; devices transmit only when asked. When a device is plugged in, the host asks it to describe itself, a process called **enumeration**. The device responds with **descriptors** that identify it.

Many devices belong to standard **classes**. A keyboard or mouse is a human interface device (HID), a USB stick is a mass storage device (MSC), and a serial adapter is a communications device (CDC). A host that supports a class works with every device of that class without additional software. As a result, a USB keyboard from any manufacturer works on any computer, and a device that

uses a standard class needs no drivers. This is particularly valuable for long-term storage devices: one that appears as a standard drive can be read by any computer that supports the class, now or in the future.

Each USB connector also carries 5 V power, from which many small devices, and some small computers, are powered.

7.9 Video

A video display draws its picture as a **raster**: a beam, or its modern equivalent, sweeps across each line from left to right and steps down line by line from top to bottom, then returns to the top and begins again. The signal carries the colour of each pixel in turn, together with **sync** pulses that mark the end of each line and of each frame.

The standard VGA mode of 640 by 480 pixels at sixty frames a second has a pixel clock of about 25 MHz. Including the time spent in the margins and sync pulses, each frame occupies 800 by 525 pixel times.

A computer produces this signal from a **framebuffer**, an area of memory holding one number per pixel. A video circuit reads the framebuffer in step with the raster and converts each number into a colour. A program draws by writing to the framebuffer. In an FPGA computer, the video circuit is part of the design, so one machine can offer several screen modes and several kinds of output:

- **VGA** sends the colours as analog voltages on separate wires.
- **DVI** and **HDMI** send the same picture digitally, as fast serial streams.
- **Composite** video combines the picture into a single signal in the format of older televisions, which remain widely available.

7.10 Audio

Sound output is the reverse of the sampling described in Information. A stream of numbers, one per sample, is converted back into a voltage by a DAC, or by a fast pulse stream and filter, and then amplified. Sixteen-bit samples at tens of thousands per second are sufficient for good sound. A **mixer** adds several streams together, so that music, speech and sound effects can play at the same time.

7.11 Keyboards and Mice

A keyboard does not send letters. It sends **scan codes**, numbers that identify which key was pressed and which was released. The computer translates them into characters according to a **keyboard layout**, so the same key produces a Z under one layout and a Y under another. Holding Shift, or a key such as AltGr, adds further state to the translation.

A mouse reports small movements, the distance moved left or right and up or

down since the last report, together with the state of its buttons. The computer adds up the movements to determine the position of the pointer.

7.12 Connectors and Modules

Expansion connectors allow a small computer to be extended. Many FPGA boards use **Pmod** connectors: rows of six pins, with four signals plus ground and 3.3 V, often doubled to twelve. Because the connector is simple, any of the buses in this chapter fits on a Pmod row, and the same socket can accept a memory module, a network adapter, a sensor, a display or a programmable module.

This is the modularity principle from Timeless Computing in practice. When a new need arises, a small, inexpensive module meets it, and the computer itself remains unchanged.

8 Operating Systems

A processor runs one instruction at a time, from one program. A desktop computer nevertheless appears to run many programs at once, shares one screen and one disk among them, and prevents a faulty program from stopping the others. The software that does this is the **operating system** (OS). This chapter describes what an operating system does and the main choices in designing one.

8.1 When an Operating System Is Needed

Not every computer has an operating system. A small module that reads a sensor and switches a pump runs a single program, written for its hardware, from the moment it is powered on. This is called running on **bare metal**. For one task on one small device it is often the better choice, because there is less to go wrong and less to understand.

An operating system is needed when a computer must run several programs, share devices among them, store files, and be used by a person for many different tasks. It has four broad functions:

- **Sharing** the processor, memory and devices among programs.
- **Abstraction** of the hardware, so that a program can write a file without knowing how the SD card works.
- **Protection** of programs from each other, and of the system from them.
- **Services** that most programs need, such as files, windows, networking and time.

8.2 Booting

When power is applied, a processor starts fetching instructions from a fixed address. The program at that address must be present before anything else has happened, so it is kept in non-volatile memory. This first program, the **bootloader** (on PCs, part of the firmware known as the BIOS), sets up the hardware, finds the operating system, copies it into memory and jumps to it. The core of the operating system, the **kernel**, then starts its drivers and finally the first program the user sees: a command prompt, a desktop or an application.

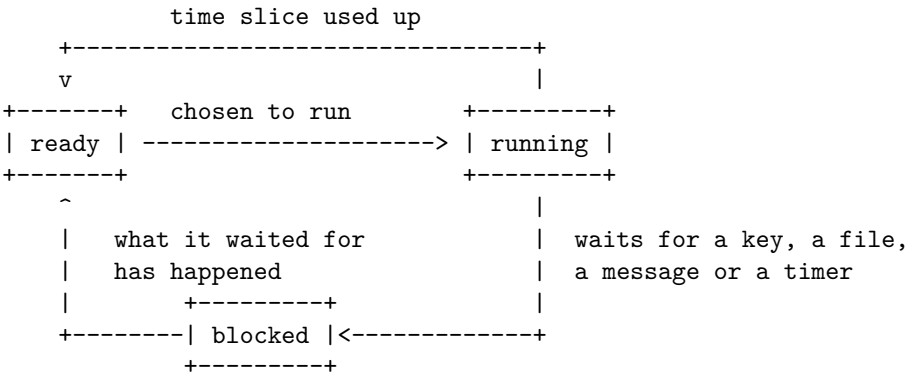
An FPGA computer has one additional step at the beginning. Before there is a processor, the FPGA must load its own configuration, the design that makes it a computer, from a flash memory chip. Only then does the processor defined by that design begin to fetch instructions.

The boot sequence is also the usual means of recovery. A well-designed system offers a way to interrupt booting, usually by holding a key, and to start something simpler and more reliable instead: a minimal console, a known good copy of the system, or a way to load a new one. This is the recoverability principle from Timeless Computing. The owner of any computer should know how to reach it.

8.3 Processes

A program on disk is a file. A program that is running is a **process**: its code and data in memory, its stack, the contents of its registers, and the resources it is using, such as open files. The kernel keeps a table of processes, each identified by a number, its process ID.

At any moment, each process is in one of a few states:



- **Running**: on the processor now. With one processor, only one process is running.
- **Ready**: able to run, and waiting for its turn.
- **Blocked**: waiting for something, such as a key press, data from a disk or a message, and not using the processor.

Most processes spend most of their time blocked. A text editor does nothing until a key is pressed. For this reason one slow processor can serve many programs, since only a few need it at any moment.

8.4 Scheduling

The part of the kernel that decides which ready process runs next is the **scheduler**.

In **cooperative** multitasking, each program runs until it voluntarily gives up the processor. This is simple, but one program that never yields, whether through a bug or poor design, stops the whole system.

In **preemptive** multitasking, a timer interrupt, described in The Computer, takes the processor away from the running program at regular intervals. The kernel saves that program's registers, chooses another ready process, restores its registers and lets it run. This exchange is a **context switch**. Each process runs for a short **time slice**, typically about a millisecond. Because this happens hundreds of times a second, every program appears to run continuously.

A scheduler can aim for **throughput**, the most total work done, or for **responsiveness**, a quick reaction when something happens. For a computer used

by a person, responsiveness matters most. A well-designed scheduler runs a process as soon as the event it was waiting for arrives, so that a key press is handled at once, even on a slow processor. Speed and responsiveness are different properties, and a slow machine can be the more responsive of two.

8.5 Memory and Protection

The kernel decides where in memory each process is placed, and allocates more memory to processes that request it.

It must also prevent one process from damaging another. A program with a bug might write through a bad address into another program's memory, or into the kernel. Without protection, one bug can stop the whole machine.

Processors provide different levels of support:

- **None.** Every program can read and write all of memory. Small and early systems worked this way. It is simple and fast, but one faulty program can corrupt everything.
- A **memory protection unit** (MPU) lets the kernel mark ranges of memory as readable, writable or executable for the running process. A process that accesses memory outside its own is stopped and reported, and the other processes continue.
- A **memory management unit** (MMU) also translates every address a program uses into a different physical address, through tables kept by the kernel. Each process then has its own private view of memory, called **virtual memory**, and parts of it can be moved to disk when memory runs short. Large systems such as Linux depend on an MMU.

Protection also requires **privilege levels**. The kernel runs at a higher privilege than ordinary programs, and only privileged code can change the protection settings. Otherwise a program could switch the protection off.

8.6 System Calls

Because ordinary programs are not permitted to do everything, they ask the kernel to act for them. A **system call** is such a request: to open a file, read some bytes, send a message, start a program or wait for an event. Each operating system has its own set of system calls, and the standard library of each programming language provides them as convenient functions.

8.7 Drivers

A **driver** is the part of the system that operates one particular device. It knows which registers to write, in what order, and what the replies mean. The rest of the system uses the device through a simpler, general interface. To a program writing a file, an SD card, a USB stick and a RAM disk appear the same.

8.8 Files and Names

A filesystem, described in Memory and Storage, organizes a storage device into named files in folders. The operating system joins all its storage devices into one **namespace**, the set of names a program can use.

Unix-like systems, and many systems influenced by them, use a single tree of names written as **paths**, such as `/home/notes.txt`. The leading `/` is the root of the tree. Each device is attached, or **mounted**, at some point in the tree, so that a USB stick might appear as `/usb` with its contents below that. Other systems, such as MS-DOS and Windows, give each device a letter instead.

The advantage of a single tree is that everything in it is accessed in the same way. A program that can read files can read them from any device, and some systems present devices, settings and network resources as files in the tree.

8.9 Communication Between Processes

Processes often need to work together. A text editor asks the filesystem for a file, and a window manager tells an application that a key was pressed. Operating systems provide several methods:

- **Messages**: small packets of data sent from one process to another through the kernel.
- **Pipes**: one-way streams of bytes from one process to another. Unix shells use them to chain commands, each feeding its output to the next.
- **Shared memory**: a region that both processes can read and write. It is fast, but the processes must coordinate so that neither changes data the other is using.

8.10 Kernel Designs

The largest design decision in an operating system is how much belongs inside the kernel.

A **monolithic kernel** contains the scheduler, memory management, filesystems, drivers and networking in one privileged program. Linux is monolithic. Every part can call every other part directly, which is fast, but a bug in any driver can crash the whole system.

A **microkernel** contains only the essentials: scheduling, memory, messages and interrupts. Filesystems, drivers and networking run as ordinary processes, which communicate with each other and with applications through messages. A driver that crashes can be restarted like any other program, and each part is small enough to understand on its own. The cost is the time spent passing messages, which careful design keeps small.

Zeitlos, described in The Machine, is a microkernel.

8.11 User Interfaces

The part of the system that people see is a program like any other.

A **command shell** reads commands typed by the user, runs them and shows the results. It is a simple and capable interface, and it can be used over a serial line or a network as easily as on the computer's own screen. A program that displays a text screen of this kind, for a local or remote shell, is a **terminal**.

An interpreter for a programming language can serve as a shell. Such an interface is called a **REPL**, for read, evaluate, print, loop: it reads an expression, evaluates it, prints the result and waits for the next. Many home computers of the 1980s started with a BASIC prompt of this kind.

A **graphical interface** has windows, a pointer and icons. A **window manager** decides where windows are placed, draws their borders and passes keyboard and mouse events to the correct application.

8.12 Simplicity as a Feature

A modern general-purpose operating system is large, because it must support thousands of devices, many users at once, and nearly every program ever written. A system for timeless applications can be much smaller. It needs to support one person, a known family of hardware, and a modest set of carefully written programs.

The benefit of a small system is that it can be understood. A system small enough to read can be trusted, repaired and changed by the person who uses it, and when something goes wrong there are fewer places to look.

9 Communication

This chapter explains how computers exchange data, from a single wire between two machines to the Internet, and how they keep those exchanges private. The same principles apply at every scale.

9.1 A Link Between Two Machines

The simplest network is two computers joined by a serial cable, as described in Talking to the World. Each byte sent by one arrives at the other. Even this requires some agreement between the two: the same speed, the same way of marking where one message ends and the next begins, and some way to detect a byte that arrives damaged.

The first computer terminals were teleprinters, typewriters that sent each key press over a wire and printed whatever came back. Their screen-based successors added **escape sequences**: runs of characters, starting with the escape code, that move the cursor, clear the screen or change colours instead of printing. The sequences used by the DEC VT100 terminal of 1978 became a standard that nearly every terminal program still understands. A terminal program on a modern computer is in effect a simulated VT100.

For decades, people connected computers over the telephone network with **modems**, which convert bits into sounds and back. A common use was the **bulletin board system** (BBS): a computer that others could dial into, one or a few callers at a time, to read and post messages, exchange files and play games. Many BBSes now run on the Internet and are reached with telnet or SSH, programs that open a terminal on another computer across a network. They remain a text-based way for a community to meet.

9.2 Packets

A dedicated wire works for two machines, but connecting many machines this way is impractical, so they share connections instead. The idea that made modern networks possible is to divide every message into **packets**: short blocks of data, each preceded by a **header** that gives its source, its destination, its length and a checksum to detect damage. Packets from many conversations can then share the same wires, taking turns, and each finds its own way to its destination.

9.3 Layers

Network software is organized in **layers**. Each layer uses the one below it and offers a simpler service to the one above, in the same way that a program uses the operating system without knowing how the disk works.

Layer	Function	Examples
physical and link	move frames to neighbours	Ethernet, Wi-Fi, radio
network	carry packets across networks	IP
transport	deliver data to programs	TCP, UDP
application	provide a service to the user	SSH, HTTP, IRC, DNS

A message from an application passes down through the layers, each adding its own header. It travels across the network as a **frame** (the link layer's name for a packet), and passes back up through the layers at the other end, each removing its header. As a result, each layer can change independently of the others. The same web page can travel over a cable, over Wi-Fi or over a radio link without any change to the application.

9.4 The Link Layer

Ethernet connects machines on a local network by cable. Each network interface has a unique 48-bit **MAC address**, set at the factory. Data travels in frames that carry the sender's and receiver's MAC addresses and a CRC. A **switch** joins several cables and forwards each frame only to the port where its destination is.

Wireless networks such as **Wi-Fi** perform the same function by radio. Low-power radios such as **LoRa** send small messages over many kilometres at low speed. A **mesh** of such radios, in which each one passes on messages for its neighbours, can connect a whole area with no other infrastructure.

9.5 IP: Addresses and Routing

The **Internet Protocol** (IP) gives every machine on a network an **IP address**. The older and still most common form, IPv4, uses 32 bits, written as four bytes in decimal, such as `192.168.1.20`. The newer IPv6 uses 128 bits, enough for every device that will ever exist.

Addresses are organized in networks. Machines on the same local network can reach each other directly. To reach anything else, a machine sends the packet to a **router**, its **gateway**, which forwards it to another router closer to the destination, and so on, hop by hop, until it arrives. No single router knows the whole route. Each knows only which way to send packets for each part of the address space. For this reason the Internet can grow without any one party managing all of it.

Two services make IP easier to use:

- **DHCP** assigns addresses automatically. A new machine asks the local network for an address, and a DHCP server assigns one and tells it which gateway to use.

- **DNS**, the domain name system, translates names such as `machdyne.com` into IP addresses by querying a hierarchy of servers.

9.6 Transport: TCP and UDP

IP delivers packets on a best-effort basis. They may arrive out of order, twice, or not at all. The transport layer determines how this is handled.

UDP sends individual messages, called **datagrams**, with no guarantees. It is simple and quick, and it suits uses where a lost message does not matter or will be requested again, such as DNS lookups, time synchronization or live audio.

TCP builds a reliable, ordered stream of bytes on top of unreliable packets. It numbers every byte it sends. The receiver acknowledges what it has received, and the sender resends anything not acknowledged in time. The receiver puts everything back in order before passing it to the program. To the program, a TCP connection behaves like a pipe: bytes go in at one end and come out at the other, all of them and in order.

Both TCP and UDP use **port numbers** to distinguish programs on the same machine. A web server listens on port 80 or 443, and an SSH server on port 22. A connection is identified by four numbers: the two IP addresses and the two ports.

9.7 Applications

The application layer provides the services that users and programs need. Many of the oldest application protocols are plain text, which makes them easy to understand and to debug by hand. A web browser requesting a page sends lines such as these:

```
GET /index.html HTTP/1.1
Host: example.com
```

The request ends with a blank line. The server replies with a status line, some headers, a blank line, and the page itself.

Some common protocols:

- **Telnet** and **SSH** provide a remote terminal. Telnet sends everything, including passwords, as plain text. SSH encrypts it.
- **HTTP** and **HTTPS** carry the web. Modern web pages are often too large for a small computer, but simple pages work well.
- **Gopher** and **Gemini** are much simpler alternatives to the web, built around text documents and menus, and suited to small machines.
- **IRC** is a long-established protocol for text chat in shared channels.
- **Email** is delivered by SMTP and collected by POP or IMAP.

9.8 Keeping Secrets

Packets pass through many machines on their way, and any of them could read or alter the data. **Cryptography** protects against this. It provides three properties:

- **Confidentiality**: only the intended receiver can read the message.
- **Integrity**: any change to the message is detected.
- **Authentication**: the receiver can be certain who sent it.

Symmetric encryption uses one secret key both to encrypt and to decrypt. It is fast, but both ends must already share the key, and delivering it to them securely is difficult.

Public-key cryptography solves that problem with a pair of keys. The **public key** can be given to anyone, and the matching **private key** is kept secret. The pair makes two things possible. Two machines that have not communicated before can agree on a shared secret over an open network, even while others are listening. In addition, a message can carry a **digital signature**, made with the private key, which anyone with the public key can check and no one without the private key can forge.

Secure protocols combine these methods. When a browser connects to an HTTPS site, or a terminal to an SSH server, the two ends use public-key cryptography to authenticate and to agree on a symmetric key. They then use that key and fast symmetric encryption for the rest of the session. The protocols are called **TLS** and **SSH**.

Authentication raises a question of trust: how to know that a public key belongs to the intended site. The web relies on **certificates**, which are public keys signed by organizations that browsers are configured to trust. SSH usually asks the user to confirm a server's key the first time and remembers it afterwards.

Cryptography also needs good **random numbers**, so that keys cannot be guessed. Computers are deterministic, so they gather randomness from physical noise. Some chips, including the FPGA designs in this book, contain a **true random number generator** that measures the jitter of free-running oscillators.

All of this requires computation. On a small processor, the arithmetic of public-key cryptography can take seconds per connection. FPGA computers have an advantage here: the slow parts can be given their own circuits in the FPGA, which makes them many times faster.

9.9 Networks Without the Internet

The Internet is convenient, but it is not the only means of communication, and it may not always be available.

- A **local network** of a switch and a few cables, or a single serial link, needs nothing from outside.
- **Radio meshes** can carry short messages across a town or a valley without any infrastructure.
- **Store and forward** systems pass messages on whenever a link is available. Bulletin boards once exchanged mail this way, calling each other at night when telephone rates were lower.
- Data can be carried by hand on storage devices, a method known as **sneakernet**. It can still move large amounts of data faster than most networks.

A timeless computer should be useful with no network at all, and able to use whatever network is available when there is one. Plain text, simple protocols and open standards make both possible.

This completes the description of the machine: a processor, memory, devices, an operating system and a network. The next part of the book is about programs.

Part IV

Programming

10 Thinking in Programs

A program is a precise description of how to perform a task, written for a machine that does exactly what it is instructed to do and nothing else. Learning to program consists mostly of learning to divide a task into steps small and clear enough for a machine to follow. This chapter covers the ideas common to every programming language. The three chapters after it show those ideas in three different languages, each of which runs on Machdyne's systems.

10.1 Why Program

A computer can be used without being programmed. Programming, however, is what makes a general-purpose computer general. A person who can write short programs can make the machine perform tasks that nobody planned for: renaming a thousand files, logging a temperature every hour, converting a document or controlling a pump.

Programming also teaches how computers work, which makes faults easier to find and correct. Code generated automatically, including by AI models, must still be read, understood and corrected by someone. In addition, a person who can program can keep their tools working without depending on an outside service.

10.2 Algorithms

An **algorithm** is a method for solving a problem in a finite number of exact steps. Algorithms are older than computers. One of the oldest known, recorded by Euclid around 300 BC, finds the greatest common divisor of two numbers, the largest number that divides both exactly:

```
to find the greatest common divisor of A and B:
    while B is not zero:
        let R be the remainder of A divided by B
        replace A with B, and B with R
    the answer is A
```

For 1071 and 462, the remainders are 147, then 21, then 0, so the answer is 21. The appendix Three Languages Side by Side shows the same algorithm in BASIC, Scheme and C.

A description of steps in plain words, such as this one, is called **pseudocode**. It is a useful way to plan a program before choosing a language.

10.3 State

A program works by changing **state**: the values held in memory. A **variable** is a named place that holds a value. Assigning to a variable replaces its previous value:

```
count = 0
count = count + 1
```

The second line is not an equation, which would be false. It is an instruction to compute `count + 1` and store the result back in `count`. Every step of a running program can be understood as reading some state and writing some new state. When a program behaves incorrectly, the question is almost always which part of the state differs from what was expected, and when it changed.

Values have **types**: whole numbers, fractional numbers, characters, text, true-or-false values and others. A type determines what a value means and which operations apply to it. As Information showed, the same bits can mean different things. Types are the means by which a language records which meaning applies.

10.4 Control Flow

Programs are built from three kinds of structure:

- **Sequence**: one step, then the next.
- **Choice**: if a condition is true, one step; otherwise another.
- **Repetition**: a step repeated a fixed number of times, or until a condition becomes true.

Every algorithm, however large, can be written with these three. Euclid's method above uses all of them: a sequence of steps inside a loop that repeats while a condition holds.

10.5 Functions

A **function**, also called a procedure or subroutine, is a named part of a program that takes some inputs, called **arguments**, performs a task, and usually returns a result. Once written, a function can be used without regard to how it works. It is the most important tool in programming, because it is the means by which a large problem is divided into small ones.

A good function does one clear thing and has a name that describes it. A program for a weather station might have functions to read the sensor, convert the reading to degrees, decide whether it is freezing and write a line to the log. Each is short and can be checked on its own, and the main program reads almost as a description of the task.

A function can call itself. This is **recursion**, and it suits problems that contain smaller instances of themselves. The factorial of a number n, written n!, is n times the factorial of n - 1, and the factorial of 0 is 1. A recursive function states exactly that. Recursion needs a **base case**, here 0, at which it stops. Without one, the function calls itself until the stack is exhausted.

10.6 Data Structures

The arrangement of data matters as much as the steps that operate on it. A few structures cover most needs:

- An **array** is a row of values of the same type, numbered from 0, each reached directly by its number. Arrays are fast and simple, but fixed in size.
- A **string** is a sequence of characters, usually stored as an array.
- A **record**, or structure, groups a few named values that belong together, such as a name, a date and an amount.
- A **linked list** is a chain of records, each holding a value and the address of the next. It can grow and shrink easily, but reaching the hundredth item requires passing through the first ninety-nine.
- A **tree** is similar to a list, except that each record can point to several others. A folder of files, in which each folder can hold further files and folders, is a tree.
- A **hash table**, also called a dictionary or map, stores values under keys, such as words and their definitions, and finds a key quickly without searching every entry.

With the right structure, the algorithm is often straightforward.

10.7 Efficiency

On a slow computer, the efficiency of an algorithm has practical consequences. The useful measure is how the amount of work grows as the amount of data grows.

Finding a name in an unsorted list of n names may require checking all n of them. If the list doubles in length, the work doubles. In a sorted list, a program can instead examine the middle entry, determine which half contains the name, and repeat. Each step halves the remaining entries, so a million names require only about twenty comparisons. This method is **binary search**, and it is the difference between a program that answers at once and one that makes the user wait.

Some methods grow much faster. Comparing every item with every other item takes n times n steps, so a thousand items require a million comparisons. A loop inside a loop over a large amount of data is often a sign that a better method exists.

10.8 Compilers and Interpreters

A processor runs only machine code, so a program in any other language must be translated. There are two approaches.

A **compiler** translates the whole program into machine code before it runs. The result runs at full speed, and many mistakes are reported before the program

runs at all. The cost is a separate step between writing and running. C is normally compiled.

An **interpreter** reads the program and carries out each part directly, as it proceeds. It is slower, often by a factor of ten or more. In exchange, a programmer can type a line and see its effect immediately, change the program while it runs, and work on a machine that has no compiler. BASIC and Scheme are usually interpreted.

Many systems lie between the two. Some interpreters first translate the program into a compact internal form, often called **bytecode** or tokens, and interpret that, which is faster than working from the text. Machdyne BASIC checks each line and converts it into tokens as it is typed.

10.9 Mistakes

Almost every program contains mistakes, called **bugs**, when first written. They are of three broad kinds:

- **Syntax errors**: the program is not valid in its language, in the way that a sentence with its words in the wrong order is not valid. The compiler or interpreter reports these.
- **Run-time errors**: the program is valid but attempts something impossible when it runs, such as dividing by zero or reading past the end of an array.
- **Logic errors**: the program runs without complaint and gives the wrong answer. These are the most difficult to find, because nothing reports them.

Finding bugs is called **debugging**. The method matters more than the tools:

1. **Reproduce** the problem reliably. A bug that cannot be made to occur on demand cannot be confirmed as fixed.
2. **Observe** instead of guessing. Print the values of variables at key points, or step through the program, and compare the results with what was expected.
3. **Narrow the search.** Find the earliest point at which the state is wrong. Halving the search space each time, as in binary search, finds the cause quickly.
4. **Fix the cause**, not the symptom. Then check that the original problem is gone and that nothing else has broken.

Small **tests**, programs that run a function with known inputs and check its outputs, find many bugs early and prevent them from returning.

10.10 Writing for People

A program is read far more often than it is written, frequently by its own author months later. Clear names, short functions, consistent layout and brief

comments that explain why something is done, rather than what, make a program much easier to maintain. The simplest program that performs the task is usually the best one.

10.11 Three Languages

The next three chapters use three languages. Each represents a different way of thinking about programs, and each has been in use for decades. All three run on Zeitlos, and BASIC also runs on Machdyne's modules.

Language	Born	Style	Best for
BASIC	1964	numbered lines	first programs, modules
Scheme	1975	expressions and functions	ideas, structure, scripting
C	1972	near the machine	speed, hardware, systems

The chapters can be read in any order, starting with the language that best suits the reader's purpose.

11 BASIC

BASIC was created in 1964 at Dartmouth College by John Kemeny and Thomas Kurtz, so that students in any subject, not only scientists, could use a computer. Its name stands for Beginner's All-purpose Symbolic Instruction Code. In the late 1970s and 1980s, almost every home computer started with a BASIC prompt, and many people learned to program by typing in listings printed in magazines.

BASIC remains useful. An interpreter fits in a few kilobytes, so it runs on the smallest chips. A program is a short list of numbered lines that is easy to read, and the result of each change is seen at once. This makes it a good first language and a practical one for small machines.

11.1 Machdyne BASIC

There are hundreds of dialects of BASIC. This chapter uses **Machdyne BASIC 1**, which runs on Machdyne's programmable modules and tools, inside Zeitlos, and on ordinary computers. It is deliberately small, because it must fit, together with its own filesystem, in a microcontroller with 16 KB of program memory and 2 KB of RAM.

Its main features and limits:

- Programs are numbered lines, stored and run in order of line number.
- There are 26 variables, `A` to `Z`, each holding one number.
- Every number is a 16-bit signed integer, from -32768 to 32767. There are no fractions and no string variables; text appears only in quotes.
- Programs and data are stored as plain text files.
- On hardware with pins, BASIC can read and drive them directly.

Larger BASICs add string variables (written like `N$`), arrays, fractional numbers and many more statements, but the ideas are the same. Machdyne BASIC 1 is also **frozen**: its definition will not change. Later versions may add to it, but must not change the meaning of any BASIC 1 program. A program written for it today is intended to keep its meaning for decades, on whatever machine implements it then.

11.2 Lines and Commands

A line that starts with a number is stored as part of the program. Typing a line with a number that already exists replaces that line, and typing the number alone deletes it. Line numbers are traditionally written in steps of ten, which leaves room to insert lines later.

A line without a number is a **command**, and is carried out at once:

Command	Effect
RUN	run the program
LIST	show the program
NEW	erase the program
SAVE name	save the program to a file
LOAD name	load a saved program
DIR	list the files
TYPE name	show the contents of a file
DEL name	delete a file

Many BASICs also have an **immediate mode**, in which a statement typed without a line number, such as `PRINT 2 + 2`, runs at once. Machdyne BASIC 1 does not; statements must be placed in numbered lines.

11.3 A First Program

```
10 FOR I = 1 TO 5
20 PRINT I; " SQUARED IS "; I * I
30 NEXT I
```

After these lines are typed, `RUN` prints:

```
1 SQUARED IS 1
2 SQUARED IS 4
3 SQUARED IS 9
4 SQUARED IS 16
5 SQUARED IS 25
```

`FOR` and `NEXT` form a loop: `I` takes the values 1 to 5, and the line between them runs once for each value. `PRINT` shows a mixture of numbers and quoted text, separated by semicolons. A comma in place of a semicolon moves to the next print zone, 14 characters wide, which is useful for tables.

11.4 Numbers and Arithmetic

BASIC uses the operators `+`, `-`, `*` and `/`, and `MOD` for the remainder after division. Multiplication and division are done before addition and subtraction, and parentheses group as in ordinary arithmetic.

Because every number is an integer, division discards the fraction: `7 / 2` is 3. Because numbers are 16 bits, results beyond 32767 wrap around, as described in Information. A program that works with large values must scale them, for example by keeping temperatures in tenths of a degree or money in cents, and must check that intermediate results stay in range.

Comparisons, such as `A > B`, give -1 for true and 0 for false. Any value other than 0 counts as true, so a variable that holds 1 or 0 can be used directly as a

condition.

11.5 Decisions and Loops

`IF` runs the rest of its line only if a condition is true, and `ELSE` gives an alternative. `GOTO` continues the program at another line. With these and `FOR`, any control flow can be written.

This program prints the prime numbers up to 50, those divisible only by 1 and themselves:

```
10 REM PRINT THE PRIME NUMBERS UP TO 50
20 FOR N = 2 TO 50
30 P = 1: D = 2
40 IF D * D > N THEN GOTO 70
50 IF N MOD D = 0 THEN P = 0
60 D = D + 1: GOTO 40
70 IF P THEN PRINT N; " ";
80 NEXT N
90 PRINT
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

For each number `N`, the program assumes that it is prime (`P = 1`) and then tries dividing it by each `D` from 2 upward. Divisors come in pairs, and the smaller of a pair cannot exceed the square root of `N`. The search can therefore stop once `D * D` exceeds `N`, which removes most of the work. `REM` starts a remark, a comment for the reader, and a colon separates several statements on one line.

In Machdyne BASIC, the body of a `FOR` loop always runs at least once, even if the loop's end value has already been passed. For this reason the inner loop is built from `IF` and `GOTO` rather than `FOR D = 2 TO N - 1`, which would wrongly test 2 against itself.

11.6 Subroutines

Lines that perform one task can be made into a **subroutine**. `GOSUB` jumps to it, and `RETURN` continues at the statement after the `GOSUB`:

```
10 FOR N = 1 TO 3
20 GOSUB 100
30 NEXT N
40 END
100 REM PRINT A LINE OF N STARS
110 FOR S = 1 TO N
120 PRINT "*";
130 NEXT S
140 PRINT
150 RETURN
```

```
*
**
***
```

The `END` on line 40 is necessary. Without it, the program would continue into the subroutine after the loop finishes and reach `RETURN` with nowhere to return to.

Because all variables are shared, a subroutine communicates through them: here the main loop sets `N`, and the subroutine reads it. With only 26 variables, it helps to record the purpose of each at the top of the program.

A well-organized BASIC program keeps its main loop short and near the top, and places each subroutine at its own round line number, such as 100, 200 and 300, with a `REM` line describing what it does. `GOTO` can make a program hard to follow, so it is best kept for simple loops and skips, with subroutines used for everything else.

11.7 Input

`INPUT` waits for the user to type a number:

```
10 INPUT "HOW MANY"; N
```

The quoted prompt is shown first, followed by a question mark. Everything a BASIC program prints and reads passes through the **console**, which may be a screen and keyboard, a terminal on another computer, or a link over the module bus.

11.8 Files

A program can write and read plain text files. Only one file can be open at a time, as number 1:

```
10 PINS -, -, AIN, -
20 OPEN "LOG.TXT" FOR APPEND AS #1
30 PRINT #1, ADC(3)
40 CLOSE #1
50 SLEEP 60
60 GOTO 20
```

Every minute (`SLEEP` waits a number of seconds), this program reads an analog input and adds the reading as a new line at the end of `LOG.TXT`. Opening a file `FOR OUTPUT` replaces it instead, and opening it `FOR INPUT` allows it to be read with `INPUT #1`. The command `TYPE LOG.TXT` shows the result. Because the file is plain text, any computer can read it later.

On Machdyne's modules, files are stored in F-RAM or EEPROM. The filesystem is designed so that a power cut at any moment leaves every file either as it was

or as it was meant to become; at worst, a line being appended is lost. As a result, a data logger can be switched off at any time without losing its log.

11.9 Pins

On hardware that has pins, a BASIC program can read and drive them. First, the program declares the purpose of each pin with `PINS`, one mode per pin:

Mode	Meaning
-	not used
IN	digital input
PP	push-pull output
OD	open-drain output
AIN	analog input
I2C	the I2C bus (two pins together)
NET	left to the system, for the bus that connects it

`OUT` then sets an output pin high or low, `IN` reads a digital input as 0 or 1, `ADC` reads an analog input as a number from 0 to 1023, and `LED` turns the board's LED on or off. Because pins are declared first, a mistake such as driving a pin declared as an input produces an error message instead of unexpected behaviour.

This program keeps a heater on while a temperature sensor reads low:

```
10 REM KEEP A HEATER ON WHILE IT IS COLD
20 PINS -, -, AIN, PP
30 T = ADC(3)
40 IF T < 400 THEN OUT 4, 1 ELSE OUT 4, 0
50 REG 0, T / 4
60 WAIT 1000
70 GOTO 30
```

`WAIT` pauses for a number of milliseconds, so the loop runs about once a second. The program follows the pattern of nearly every control program: **read, decide, act, wait, repeat**. Pin 3 reads the sensor, and pin 4 switches the heater through a transistor or relay, as described in Talking to the World.

A practical controller would improve on this in one respect. When the temperature stays close to the threshold, the heater switches on and off every second, which wears out relays. The usual remedy is **hysteresis**: the heater switches on below one value and off only above a slightly higher one, so that small fluctuations have no effect.

11.10 Talking to a Host

Line 50 above stores a value in **program register** 0. A program has sixteen registers, `REG 0` to `REG 15`, each holding a byte, which are shared with the

system the program runs in. On a module, a host computer on the bus can read and write them while the program runs. In this example, the host can read the latest temperature at any time without stopping the program, and could set a register to pass in a new threshold.

11.11 Starting by Itself

A program saved as `BOOT` runs automatically when the machine starts. This turns a module into a dedicated controller: once programmed and installed, it performs its task every time it is powered, with or without a computer attached. A module always gives a host a short period after power-on in which to stop the program, so a faulty `BOOT` program cannot lock the host out.

11.12 One Language, Many Machines

The definition of Machdyne BASIC states what a program means; each machine determines which hardware it provides. A program that only computes and prints runs the same everywhere. A program that uses pins runs wherever those pins exist, and on a machine without them it stops with `NOT SUPPORTED` rather than behaving unexpectedly.

Machines may also add statements of their own. A BASIC with a screen may add drawing statements such as `PLOT`, `LINE` and `CIRCLE`, as the home computers did, and programs that use them run only where those statements exist.

As a result, a program can be written and tested on a full computer, within the limits of the module it is intended for, and then sent to a module the size of a fingernail, where it behaves the same way.

12 Scheme

Lisp was invented by John McCarthy in 1958, and is one of the two oldest programming languages still in use. Scheme is a small dialect of Lisp, created at MIT in 1975 by Guy Steele and Gerald Sussman. Its complete definition fits in about fifty pages, and it has been used for decades to teach the foundations of computer science.

Scheme programs are built from a few ideas combined freely: expressions, functions and lists. Learning it teaches a way of thinking about programs, including how a programming language itself works.

12.1 Machdyne Scheme

The Scheme in this chapter is Machdyne Scheme, a small interpreter that runs inside Zeitlos and on ordinary computers. It implements most of the R4RS standard of Scheme. It omits a few advanced features that are difficult to fit on a small machine, and the standard procedures for files, which each system provides in its own way. All its numbers are floating point. Everything in this chapter is standard Scheme and works in any implementation.

12.2 Expressions

Every part of a Scheme program is an **expression**, and every compound expression is written in the same way: in parentheses, with the operation first and its arguments after it.

```
(+ 1 2)              3
(* 6 7)              42
(+ 1 (* 2 3))        7
(- 10 (/ 8 2))       6
```

This **prefix notation** has two advantages. There are no precedence rules, because the parentheses state exactly what is grouped with what. In addition, operations can take any number of arguments: `(+ 1 2 3 4)` is 10.

Scheme is normally used through a **REPL** (read-eval-print loop), which reads an expression typed by the user, prints its value, and waits for the next. In this chapter, `; =>` marks the value an expression produces. Everything from a semicolon to the end of the line is a comment.

12.3 Definitions

`define` gives a name to a value or to a function:

```
(define (square x) (* x x))
(square 12)                    ; => 144
```

This reads as: to compute the square of `x`, multiply `x` by `x`. Once defined, `square` is used in the same way as the built-in operations.

12.4 Choices

`if` takes a test, an expression for when the test is true, and one for when it is false. `cond` checks a series of tests in order and uses the first that is true:

```
(define (size n)
  (cond ((< n 10) 'small)
        ((< n 100) 'medium)
        (else 'large)))
(size 42)                       ; => medium
```

The values `'small`, `'medium` and `'large` are **symbols**: names used as data, an alternative to strings or numeric codes. The quote mark means "this name itself, not the value it stands for".

The true and false values are written `#t` and `#f`. In Scheme only `#f` counts as false; zero, the empty list and the empty string all count as true.

12.5 Recursion

Scheme has no statement for loops. Repetition is written as **recursion**, in which a function calls itself:

```
(define (factorial n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))
(factorial 10)                  ; => 3628800
```

The definition states what a factorial is: 1 for zero, and otherwise n times the factorial of n - 1. Its evaluation can be followed by replacing each call with its definition:

```
(factorial 3)
(* 3 (factorial 2))
(* 3 (* 2 (factorial 1)))
(* 3 (* 2 (* 1 (factorial 0))))
(* 3 (* 2 (* 1 1)))
6
```

Each pending multiplication must be remembered until the innermost call returns, so this function uses stack space in proportion to `n`.

12.6 Tail Calls

When the recursive call is the last action of a function, nothing needs to be remembered, and the call can reuse the current stack frame. Scheme guarantees

this: such a **tail call** runs in constant space, and is therefore as efficient as a loop. A loop is usually written with a **named let**, which defines a local function and calls it immediately:

```
(define (sum-to n)
  (let loop ((i 1) (total 0))
    (if (> i n)
        total
        (loop (+ i 1) (+ total i)))))
(sum-to 100)                    ; => 5050
```

`loop` starts with `i` as 1 and `total` as 0. Each call to `loop` is a tail call that carries the new values forward, so the computation can repeat a million times without growing the stack. The variables that would change in a loop in another language are here the arguments of the next call.

12.7 Lists

The **list** is the central data structure of Lisp, whose name comes from "list processing". A list is written in parentheses and quoted, so that Scheme treats it as data rather than as an expression to evaluate:

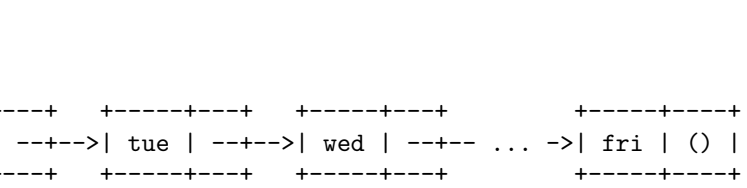
```
(define days '(mon tue wed thu fri))
(car days)                      ; => mon
(cdr days)                      ; => (tue wed thu fri)
(cons 'sun days)                ; => (sun mon tue wed thu fri)
```

Three operations do most of the work:

- `car` gives the first item of a list.
- `cdr` gives the rest of the list, everything after the first item.
- `cons` builds a new list with one item added at the front.

The names come from two parts of a machine word on the IBM 704, the computer on which Lisp was first implemented.

Internally, a list is a chain of **pairs**. Each pair holds one item and a reference to the next pair, and the chain ends with the empty list `'()`. This is the linked list described in Thinking in Programs:



`cons` does not copy the list it adds to; it makes one new pair that points to the old list. As a result, lists can share their tails, and building a list one item at a time is inexpensive.

Most functions on lists follow the same pattern: they handle the empty list, and otherwise do something with the `car` and recurse on the `cdr`:

```
(define (sum lst)
  (if (null? lst)
      0
      (+ (car lst) (sum (cdr lst)))))
(sum '(3 1 4 1 5))            ; => 14
```

12.8 Lookup Tables

A list of pairs can serve as a lookup table, called an **association list**. A pair whose second part is not a list is written with a dot:

```
(define phones '((anna . 4711) (ben . 2501) (cleo . 1234)))
(cdr (assq 'ben phones))      ; => 2501
```

`assq` finds the first pair whose first part is the given symbol, or returns `#f` if there is none. For a few dozen entries, an association list is sufficient.

12.9 Functions as Values

In Scheme, a function is a value like any other. It can be passed to another function, returned from one, or stored in a list. A function that takes or returns functions is called **higher-order**.

The standard function `map` applies a function to every item of a list:

```
(map square '(1 2 3 4))       ; => (1 4 9 16)
```

Higher-order functions are defined like any other. This one keeps the items of a list for which a test is true:

```
(define (keep ok? lst)
  (cond ((null? lst) '())
        ((ok? (car lst)) (cons (car lst) (keep ok? (cdr lst))))
        (else (keep ok? (cdr lst)))))

(keep even? '(1 2 3 4 5 6))           ; => (2 4 6)
(keep (lambda (x) (> x 10)) '(5 50 8 80))   ; => (50 80)
```

`lambda` makes a function without giving it a name, which is convenient when a small function is needed only once. The definition `(define (square x) (* x x))` is shorthand for `(define square (lambda (x) (* x x)))`.

With `map`, `keep` and similar functions, many tasks that need a loop and some bookkeeping in other languages become a single expression.

12.10 Closures

A function made by `lambda` retains access to the variables that were visible where it was made, even after the code that created them has finished. Such a function is a **closure**:

```
(define (make-counter)
  (let ((count 0))
    (lambda ()
      (set! count (+ count 1))
      count)))

(define tally (make-counter))
(tally)                         ; => 1
(tally)                         ; => 2
```

Each call to `make-counter` creates a new variable `count` and returns a function that can reach it; no other code can. `set!` changes a variable's value; by convention, the exclamation mark indicates that something is changed in place. Closures provide private, persistent state without any additional language features, and many larger programming techniques are built on them.

12.11 Memory

Scheme programs create pairs, strings and closures freely and never release them. The interpreter releases them automatically: from time to time, a **garbage collector** finds every object the program can still reach, starting from its variables, and recycles the rest. Scheme programmers, like those of most modern languages, never free memory by hand, which eliminates a whole class of errors. The cost is some additional memory and an occasional pause while the collector runs.

12.12 Code Is Data

A Scheme program is written as lists, and lists are Scheme's main data structure. As a result, a program can readily build, examine and run other programs. This is an evaluator for a small language of sums and products:

```
(define (calc e)
  (cond ((number? e) e)
        ((eq? (car e) '+) (+ (calc (cadr e)) (calc (caddr e))))
        ((eq? (car e) '*) (* (calc (cadr e)) (calc (caddr e))))
        (else 'unknown)))

(calc '(+ 1 (* 2 3)))          ; => 7
```

`cadr` and `caddr` are shorthand for the second and third items of a list. The evaluator examines the expression. A number is its own value. A list starting

with `+` or `*` is evaluated by evaluating its two arguments and combining the results.

Interpreters work in this way. If `calc` is extended with variables, `if`, `lambda` and function calls, it becomes an interpreter for Scheme itself, which is known to require only a page or two of code. This shows that a programming language is itself a program, and that a small one can be complete.

12.13 Scheme on a Machine

A Scheme system on a particular computer adds procedures for what the standard leaves out: files, windows, the network and the hardware. With them, a few lines of Scheme typed at the REPL can automate the machine or form a small windowed tool, without the need for a compiler.

Scheme is well suited to problems concerned with structure: lists of things, rules, transformations, small languages, and problems that benefit from interactive exploration. For obtaining the greatest speed from a slow processor, C is the better choice.

13 C

C was created by Dennis Ritchie at Bell Labs around 1972, to write the Unix operating system. It was designed to be close to the machine, so that programs could be nearly as fast as assembly language while remaining portable from one kind of computer to another. Half a century later, it is still the main language of operating system kernels, device drivers, microcontrollers and the interpreters of other languages. The Zeitlos operating system, the BASIC interpreter and the Scheme interpreter in this book are all written in C.

C shows directly what a computer does. Its variables are locations in memory, its pointers are addresses, and its operations correspond closely to single machine instructions. It also checks little: it does what the program says, and allows a faulty program to damage itself. Both are reasons to learn it well.

13.1 A Complete Program

```
#include <stdio.h>

#define BIT(n) (1u << (n))

int main(void)
{
    unsigned flags = 0;

    flags |= BIT(3);            /* set bit 3 */
    flags |= BIT(0);            /* set bit 0 */
    flags &= ~BIT(3);           /* clear bit 3 */
    flags ^= BIT(5);            /* flip bit 5 */

    if (flags & BIT(0))         /* test bit 0 */
        printf("bit 0 is set\n");

    printf("flags = 0x%02X\n", flags);
    return 0;
}
```

```
bit 0 is set
flags = 0x21
```

The parts of this program appear in nearly every C program:

- `#include <stdio.h>` makes the standard input and output functions, such as `printf`, available.
- `#define` creates a **macro**, a piece of text that is substituted wherever its name appears before compilation. Lines starting with `#` are handled by this **preprocessor** step.

- `main` is the function where every program starts. Its result, here 0, tells the system whether the program succeeded.
- Statements end with a semicolon, and braces group them into blocks.
- Text between `/*` and `*/` is a comment.
- `printf` prints formatted text. In its first argument, `%02X` means "the next value, in hexadecimal, at least two digits wide", and `\n` means a new line.

The program demonstrates C's bitwise operators, which act on every bit of a number at once: `|` (OR), `&` (AND), `^` (XOR), `~` (NOT) and `<<` (shift left). `BIT(5)` is 1 shifted left five places, a number with only bit 5 set. This is called a **mask**. OR-ing with a mask sets that bit, AND-ing with its inverse clears it, XOR-ing with it flips it, and AND-ing with it tests it. These four operations control the individual bits of a hardware register.

13.2 Compiling

A C program is written in one or more text files and turned into machine code by a **compiler**. Each source file is compiled separately, and a **linker** then joins the results, together with any **libraries** of ready-made functions, into one executable program. **Header files**, ending in `.h`, declare the functions and types that a source file or library provides, so that other files can use them.

The **standard library** supplied with C provides input and output (`stdio.h`), strings (`string.h`), memory allocation and conversions (`stdlib.h`), mathematics (`math.h`) and more. Programs that use only the standard library can be compiled for almost any computer.

Most C compilers are large programs that run on large computers, but a compiler for a useful subset of C can be small enough to run on a small machine. Zeitlos includes an experimental one, so that a program can be written, compiled and run on the FPGA computer itself. A compiler of that size cannot do everything a large one can, but it makes the machine more self-sufficient.

13.3 Types

Every variable in C has a type, declared when the variable is created, which determines how much memory it occupies and what its bits mean:

Type	Typical size	Holds
char	8 bits	one character, or a small number
short	16 bits	a whole number
int	32 bits on most systems	a whole number
long	32 or 64 bits	a whole number
float, double	32, 64 bits	a fractional number

Any of the whole-number types can be `unsigned`, which gives up negative values

in exchange for twice the positive range.

The sizes are not fixed by the language. An `int` is 16 bits on some small processors and 32 bits on most others, which matters wherever a value might overflow. Where the exact size matters, as it does for hardware registers and file formats, the header `stdint.h` provides types with the size in their names: `uint8_t`, `int16_t`, `uint32_t` and so on.

13.4 Pointers

A **pointer** is a variable that holds a memory address. Pointers are the source of both C's power and its danger.

```
    int x = 42;
    int *p = &x;            /* p holds the address of x */
    *p = 7;                 /* store 7 at that address */
    printf("x is now %d\n", x);
```

```
x is now 7
```

`&x` means "the address of `x`". The declaration `int *p` makes `p` a pointer to an `int`, and `*p` means "the `int` at the address in `p`". Storing through `p` changes `x`, because `p` points to `x`.

Pointers allow a function to change its caller's variables, allow a program to build linked structures, and allow it to reach any address, including a hardware register.

13.5 Arrays and Strings

An **array** is a sequence of values of one type, numbered from 0. In most expressions, the name of an array stands for the address of its first element, so arrays and pointers are closely related:

```
    int a[4] = { 10, 20, 30, 40 };
    int *q = a;             /* an array name is its first address */
    printf("%d %d\n", *(q + 2), q[3]);
```

```
30 40
```

Adding 2 to a pointer to `int` moves it two `int`s along, not two bytes, and `q[3]` is another way of writing `*(q + 3)`.

C has no separate string type. A **string** is an array of characters ending with a byte of zero, written `'\0'`. A function that needs the length of a string must step along it to find the end:

```
/* Count the characters in a string by walking a pointer along it
   until it reaches the zero byte that ends every C string. */
int length(const char *s)
{
```

```
    const char *p = s;
    while (*p != '\0')
        p++;
    return p - s;
}
```

`length("timeless")` is 8. `const` declares that the function will not change the characters, and subtracting two pointers gives the distance between them.

C does not check array bounds. Reading or writing past the end of an array, a common error when handling strings, uses whatever memory lies beyond it, without any warning. This kind of error, a **buffer overflow**, has been one of the most common causes of security problems. For this reason, every array access in a C program should be checked for whether it can go past the end.

13.6 Structures and Memory

A **structure** groups named values into one record. This one is a note in a linked list:

```
struct note {
    char text[32];
    struct note *next;
};
```

A C program has three kinds of memory:

- **Local variables** are stored on the stack, described in The Computer. They are created when their function starts and disappear when it returns.
- **Global variables**, declared outside any function, exist for the whole run of the program.
- The **heap** is memory requested while the program runs, with `malloc`, and returned with `free`. It lasts until it is freed.

This function adds a new note to the front of a list, using memory from the heap:

```
/* Put a new note at the front of the list, and return the new front. */
struct note *add(struct note *list, const char *text)
{
    struct note *n = malloc(sizeof *n);
    if (n == NULL)
        return list;
    strncpy(n->text, text, sizeof n->text - 1);
    n->text[sizeof n->text - 1] = '\0';
    n->next = list;
    return n;
}
```

`malloc` returns a pointer to enough unused memory for one note, or `NULL` if none is left; a correct program always checks for this. The arrow, as in `n->text`, reaches a field of a structure through a pointer. `strncpy` copies at most the given number of characters, and the line after it ensures that the text always ends with a zero byte, even if it was too long to fit. Safe C depends on details of this kind.

C has no garbage collector. Memory obtained with `malloc` must be returned with `free` exactly once, after its last use. Memory that is never freed is lost to the program; this is a **leak**. Using memory after freeing it, or freeing it twice, corrupts the heap, and the resulting faults may appear much later and in unrelated parts of the program. The usual rule is that one part of the program owns each piece of memory, and only that part frees it.

13.7 Talking to Hardware

Because C can convert any number into a pointer, it can reach a memory-mapped device register directly, as described in The Computer:

```
#define PINS (*(volatile uint32_t *)0xE0000000u)

void led_on(void)  { PINS |= 1u; }
void led_off(void) { PINS &= ~1u; }
```

The macro treats the number `0xE0000000` as the address of a 32-bit unsigned value and refers to that value, so `PINS` can be read and assigned like a variable. `led_on` sets bit 0 without changing the other bits, and `led_off` clears it.

The word `volatile` is essential. It tells the compiler that the value can change, or that writing it has effects, outside the program's control. Without it, the compiler may find that the program writes the same address twice and keep only the second write, or that it reads a status register in a loop, and read it only once. With `volatile`, every read and write in the source is performed, in order.

On a computer with memory protection, the operating system decides which device addresses each program may reach. On a microcontroller with no operating system, all of the hardware is driven in this way.

13.8 Writing C Well

A few practices prevent most problems in C:

- **Enable the compiler's warnings**, and treat them as errors. They detect many mistakes at no cost.
- **Check every result** that can fail: `malloc`, file opens, reads and writes.
- **Know the size of every buffer**, and pass it along with the pointer.
- **Keep functions short**, so that each one can be checked by reading it.

- **Test** functions individually, with known inputs, including the edge cases: empty, zero, one and the largest value.

13.9 Using the Three Together

Each language in this book has its own uses, as the table at the end of Thinking in Programs showed. They can also be combined. Many systems are built with a small, fast core in C and a more convenient language above it for scripting and experiments. When a program in BASIC or Scheme is too slow, the slow part can be rewritten in C and the rest left unchanged.

So far, the hardware under these programs has been fixed. The next part of the book describes hardware that can be changed.

Part V

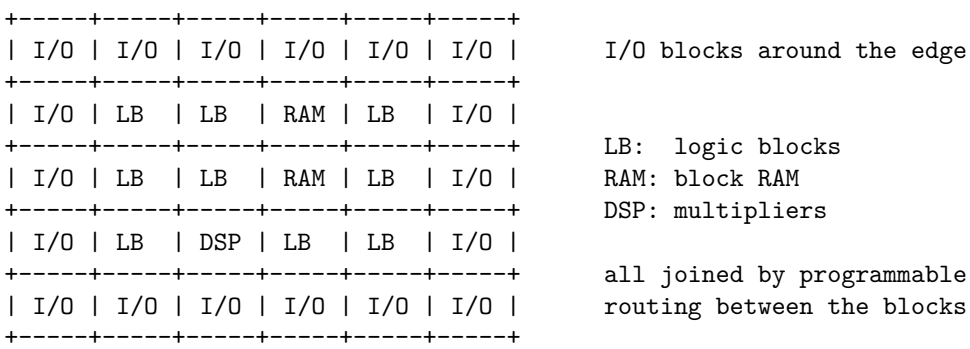
Changeable Hardware

14 Inside an FPGA

A field-programmable gate array is a chip containing many identical, uncommitted logic blocks, with programmable wiring between them. A configuration file sets up and connects those blocks to form the circuit it describes, such as a blinking light, a serial port or a complete computer. This chapter describes what is inside an FPGA and how a design becomes a configuration.

14.1 The Big Picture

An FPGA has four main kinds of component, arranged in a regular grid:



- **Logic blocks** implement the gates and flip-flops of a design.
- **Routing** connects the blocks to each other, through a mesh of wires and programmable switches.
- **Memory and arithmetic blocks** provide dedicated RAM and multipliers, which would be wasteful to build from logic blocks.
- **I/O blocks** connect the inside of the chip to its pins.

An FPGA has thousands or tens of thousands of logic blocks. Which blocks are used, how each is set up and how they are connected is held in **configuration memory** distributed throughout the chip. In most FPGAs, including all of those in Machdyne's computers, this memory is SRAM. Its contents are lost when power is removed, so the configuration must be loaded again at every start.

14.2 Look-Up Tables

The main element of a logic block is the **look-up table**, or LUT. A 4-input LUT is a memory of sixteen bits. Its four inputs form an address from 0 to 15, and its output is the bit stored at that address.

As described in Digital Logic, any function of four inputs is completely described by a truth table of sixteen rows. A LUT stores that truth table. With the appropriate sixteen bits loaded, it acts as an AND gate, an XOR gate, a multiplexer or any other function of up to four inputs, with the same delay whatever the

function. For this reason an FPGA can implement any logic design: it contains no fixed gates, only small memories that can take the place of any gate.

The sixteen bits are part of the FPGA's configuration. Design tools usually write them as one 16-bit number, often called `INIT`, whose bit 0 is the output for address 0, bit 1 the output for address 1, and so on. To make a LUT compute the XOR of inputs 0 and 1, the required output is listed for each address. For addresses 0, 1, 2 and 3, the two low bits are 00, 01, 10 and 11, so the outputs are 0, 1, 1, 0, which is binary `0110`, hex 6. The two upper inputs do not affect the result, so the same pattern repeats four times: `INIT = 16'h6666`. For an AND of all four inputs, only address 15 gives 1: `16'h8000`.

Functions of more than four inputs are divided among several LUTs. A full adder from Digital Logic, with three inputs and two outputs, requires two: one for the sum and one for the carry.

Different FPGA families use LUTs with 4 to 6 inputs, but the principle is the same.

14.3 Flip-Flops and Carry Chains

Each LUT in a logic block is paired with a **flip-flop**, so that its output can be stored at a clock edge. Together they form a small piece of combinational logic followed by a register, which is the basic unit of synchronous design.

Logic blocks also contain **carry chains**: dedicated, fast connections that pass the carry from one block to the next. Adders and counters are common enough that FPGAs provide this special path rather than sending the carry through general routing. A 24-bit counter on an ECP5 FPGA, for example, uses 24 flip-flops and 12 carry cells, each of which adds two bits.

14.4 Routing

Most of an FPGA's area is occupied by **routing** rather than logic: wires of various lengths running between the blocks, and programmable switches where they cross. Each switch is controlled by one bit of configuration memory.

Much of the delay in an FPGA is caused by routing. A signal that crosses the chip through many switches takes longer than one that travels to a neighbouring block. For this reason the same design can reach different clock speeds depending on how its parts are placed, and an FPGA runs a given circuit several times more slowly than a custom chip, in which the connections are direct metal wires.

14.5 Block RAM and DSP Blocks

Memory built from LUTs and flip-flops would use them up quickly. FPGAs therefore include **block RAM**: dedicated SRAM blocks of a few kilobits each, distributed across the chip. A design uses them wherever it needs fast on-chip

memory, such as a processor's caches and boot program, a screen font or buffers for incoming data. An FPGA with fifty-six 18-kilobit blocks has about 126 KB of block RAM, which is small compared with external memory but much faster.

DSP blocks are dedicated multipliers, often with adders attached, used for arithmetic in signal processing, graphics and cryptography. A multiplier built from LUTs would be large and slow; a DSP block performs a multiplication in one clock cycle.

14.6 I/O Blocks

Each pin of an FPGA has an **I/O block** that can be configured as an input, an output or both. The configuration sets its voltage standard, its drive strength, whether it has a pull-up resistor, and whether it captures data on a clock edge at the pin itself. Pairs of pins can be used together for **differential** signals, which carry fast data on two wires with opposite polarity and are used for HDMI and other high-speed links. Some FPGAs have dedicated high-speed serializers at their pins for this purpose.

Every I/O block belongs to a **bank** of pins that share a supply voltage. The board designer chooses that voltage, commonly 3.3 V, and every pin in the bank must use it.

14.7 Clocks

FPGAs contain **global clock networks**: routing designed to deliver a clock signal to every flip-flop on the chip at very nearly the same time. Ordinary routing cannot do this.

They also contain one or more **phase-locked loops** (PLLs), circuits that take a clock from an external crystal and produce clocks at other frequencies. A board with a 12 MHz crystal can use a PLL to produce a 48 MHz clock for its processor and a 25 MHz clock for its video output.

14.8 Configuration

The file that sets every LUT, flip-flop, switch and pin is called a **bitstream**. It is produced by the tools described in Describing Hardware and is typically between a few hundred kilobytes and a few megabytes in size.

At power-on, an FPGA reads its bitstream from a small SPI flash chip beside it, which takes a fraction of a second. The circuit it describes starts to run only after this. For testing, a bitstream can also be sent directly from another computer over a USB cable, in which case it remains until the power is turned off.

On some boards, including many of Machdyne's, the flash chip is on a small removable module. Replacing the module replaces the whole design of the computer, so the same board can be one kind of computer with one module and

an entirely different one with another. The flash can hold more than one bitstream, and some FPGAs can be instructed to reload themselves from a different address, so that a running design can pass control to another.

14.9 FPGA Capacity

FPGA capacity is usually stated in LUTs. Boards for hobbyists and small computers, including Machdyne's, use chips with a few thousand to tens of thousands of LUTs. Typical sizes of designs are:

Design	LUTs, roughly
blinking an LED	a dozen or two
a serial port	a few hundred
a small 32-bit RISC-V processor	1,000 to 3,000
the complete Zeitlos computer	about 19,000

The full Zeitlos system, with its processor, caches, memory controller, graphics, USB, audio and cryptographic accelerators, fills about three quarters of a 25,000-LUT FPGA and less than half of a 45,000-LUT one, which leaves room for additions.

14.10 FPGA Families and Open Tools

Several companies make FPGAs. Several families have open-source tools, written by people who studied the chips and documented their configuration formats:

- **Lattice iCE40**: small, simple, low-power FPGAs, the first to have a complete open toolchain.
- **Lattice ECP5**: larger FPGAs with DSP blocks and ample block RAM, the main family in Machdyne's computers.
- **Xilinx (AMD) 7-series**: large, fast FPGAs such as the Artix-7, with growing open support.
- **Cologne Chip GateMate**: FPGAs designed and made in Germany, with open-source synthesis from the start and now a fully open toolchain.

Open tools are important for timeless computing. A design that can be built only with one company's licensed software can be maintained only while that software is available and runs on current computers. A design built with open tools, whose own source code can be rebuilt, can be maintained indefinitely.

14.11 FPGAs, Processors and Custom Chips

An FPGA can be compared with a microcontroller (MCU), which is a small processor chip with built-in devices, and with a custom chip, or ASIC (application-specific integrated circuit):

	MCU	FPGA	ASIC
What it is	fixed CPU, devices	programmable logic	fixed logic
Changed by	new software	new bitstream	a new chip
Speed	modest	modest	highest
Power	lowest	low to moderate	lowest per job
Cost each	lowest	moderate	lowest in bulk
Start-up cost	very low	low	enormous

A microcontroller is suitable when its built-in processor and devices fit the task; most small controllers, including Machdyne's modules, are microcontrollers. A custom chip is suitable when millions will be made and the design will not change. An FPGA is suitable when the hardware itself must be designed, studied or changed: for parallel processing, unusual interfaces, recreations of old machines, and computers whose owners can inspect and improve the processor.

An FPGA also protects a design against obsolescence. A processor design written in a hardware description language is not tied to any one chip. When a particular FPGA goes out of production, the design can be moved to another, with the adjustments described in Moving a Design to Another Board. In this way the computer can outlast the chip it was first built on.

15 Describing Hardware

Circuits were once designed by drawing them. Digital circuits of any size are now described in text, in a **hardware description language** (HDL), and tools convert the description into a chip or an FPGA configuration. This chapter introduces Verilog, the HDL used for the designs in this book, and the steps from a description to a working circuit.

15.1 A Different Kind of Language

Verilog resembles C, but it is not a programming language in the usual sense, and this is the most important point about it. A program is a list of steps carried out one after another. A Verilog description is a list of pieces of hardware that all exist, and all operate, at the same time.

The line `assign sum = a ^ b;` does not compute a value once. It places an XOR gate whose output is always the XOR of its inputs, for as long as the circuit has power. The order of such lines in the file does not matter, as the order of parts on a circuit diagram does not.

Two languages are in common use: **Verilog**, created in 1984, and **VHDL**, from the same period. Both are standardized and widely supported, and the ideas of one apply directly to the other.

15.2 Verilog Modules

A Verilog design is built from **modules**, each with named **ports** for its inputs and outputs. The full adder from Digital Logic is written as follows:

```
module full_adder (
    input  wire a, b, cin,
    output wire sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

A **wire** is a connection, carrying whatever drives it. Each `assign` statement is a piece of combinational logic driving a wire, written with the same operators as the Boolean algebra in Digital Logic. On an ECP5 FPGA, this module becomes two LUTs.

A module is used inside another by creating an **instance** of it, as a circuit board can carry several copies of one chip. A 32-bit adder could be made of 32 full adder instances, and a computer of thousands of instances of hundreds of modules.

15.3 Numbers and Vectors

A port or wire can carry many bits at once. `wire [7:0] data` declares an 8-bit **vector**, with bits numbered 7 (most significant) down to 0. Single bits and ranges are selected with brackets, so `data[0]` is the lowest bit and `data[7:4]` the top four.

Numbers are written with their width and base: `8'd200` is the 8-bit decimal number 200, `4'b1010` is four bits in binary, and `32'hE000_0000` is 32 bits in hex, with an underscore for readability. Braces join vectors together: `{4'b0000, data[7:4]}` is an 8-bit value whose low four bits are the top of `data`.

15.4 Registers and Clocks

Combinational logic has no memory. Flip-flops and registers are described with an `always` block that runs at each rising clock edge:

```
// Blink an LED from a fast clock by counting clock cycles.
module blink #(
    parameter WIDTH = 24            // 2^24 cycles at 12 MHz: 1.4 s
) (
    input  wire clk,
    output wire led
);
    reg [WIDTH-1:0] count = 0;

    always @(posedge clk)
        count <= count + 1;

    assign led = count[WIDTH-1];    // the top bit changes slowest
endmodule
```

`reg [WIDTH-1:0] count` declares a 24-bit value. Anything assigned inside an `always` block must be declared as a `reg`, whether or not it becomes a hardware register; this one does. `always @(posedge clk)` means "at every rising edge of `clk`", and `count <= count + 1` loads the register with its value plus one. This is a counter built from 24 flip-flops and an adder, as in Digital Logic. Its top bit changes every 2 to the 23rd cycles, which with a 12 MHz clock is about every 0.7 seconds, so the LED blinks slowly enough to be seen.

`WIDTH` is a **parameter**, a constant that can be set differently for each instance. A testbench can create a 4-bit version, which blinks after a few cycles instead of millions, and check it quickly in simulation.

15.5 Two Kinds of Assignment

Inside an `always` block, Verilog has two assignment operators, and confusing them is the most common beginner's mistake.

The **non-blocking** assignment `<=` describes a register loading at the clock edge. All non-blocking assignments in a clocked block take effect together, using the values from before the edge, as physical flip-flops do. The following swaps two registers correctly:

```
always @(posedge clk) begin
    a <= b;
    b <= a;
end
```

The **blocking** assignment `=` takes effect immediately, so the next line sees the new value. It is used for combinational logic written inside `always @*` blocks, where `@*` means "whenever any input changes".

The general rule is to use `<=` in clocked blocks and `=` in combinational ones, and not to mix them in one block.

15.6 State Machines in Verilog

The traffic light from Digital Logic is written as follows:

```
// The traffic light from Digital Logic, as a state machine.
module traffic (
    input  wire clk,
    input  wire reset,
    input  wire done,                // the timer has run out
    output wire green, yellow, red
);
    localparam GREEN = 2'd0, YELLOW = 2'd1, RED = 2'd2;

    reg [1:0] state;

    always @(posedge clk) begin
        if (reset)
            state <= RED;
        else if (done)
            case (state)
                GREEN:   state <= YELLOW;
                YELLOW:  state <= RED;
                default: state <= GREEN;
            endcase
    end

    assign green  = (state == GREEN);
    assign yellow = (state == YELLOW);
    assign red    = (state == RED);
endmodule
```

`localparam` gives names to the state numbers. The `always` block is the state register together with its next-state logic. On reset, the state becomes red; otherwise it advances when the timer is done and is unchanged when it is not. The `case` statement selects by the current state. The outputs are combinational, each true only in its own state. This module and the table in Digital Logic describe the same behaviour.

Every state machine needs a **reset**, a defined state to start from. An FPGA gives its flip-flops known values when it is configured, but a custom chip does not. In addition, a simulator shows unknown values until something sets them, and a running circuit sometimes needs to start again without being reloaded.

15.7 Simulation

Hardware is difficult to debug once it is built, because a value inside a chip cannot be printed. Designs are therefore first tested by **simulation**, in which a program computes what the circuit would do, cycle by cycle, and every signal can be inspected.

A simulation is driven by a **testbench**. This is a Verilog module that describes no hardware, but is a script that creates an instance of the design, supplies its inputs and checks its outputs. Testbenches may use features that cannot become hardware, such as delays, loops over test values and printed messages. The main part of the full adder's testbench tries all eight input combinations and compares each result with ordinary addition:

```
for (i = 0; i < 8; i = i + 1) begin
    {a, b, cin} = i;
    #1;
    if ({cout, sum} !== a + b + cin)
        errors = errors + 1;
end
```

`#1` waits one time unit for the logic to settle. Simulators can also record every signal over time, to be viewed as **waveforms**, a display similar to that of an oscilloscope.

15.8 From Description to Bitstream

Converting a tested design into an FPGA configuration takes several steps, each done by a tool:

```
Verilog source          what the circuit does
     |
 synthesis              -> netlist: LUTs, flip-flops, RAMs, and wires
     |
 place and route        -> each part given a place on the chip,
     |                     each wire a path through the routing
```

```
bitstream generation   -> the configuration bits
    |
programming            -> loaded into the FPGA or its flash
```

Synthesis reads the HDL, determines the logic it describes, simplifies it, and maps it onto the building blocks the FPGA provides. The result, a **netlist**, lists every LUT and flip-flop and how they connect. Yosys is a widely used open synthesis tool.

Place and route decides which physical block on the chip each part of the netlist will use, then finds paths through the routing for every connection. It also analyses timing and reports the fastest clock the design can use. nextpnr is a widely used open place-and-route tool.

Bitstream generation, called packing by some of the open tools, converts the placed and routed design into the configuration format of the particular FPGA.

Programming loads the bitstream into the FPGA, or writes it into the configuration flash so that the FPGA loads it at every start.

The tools also need a **constraints file**, which says which signal goes to which physical pin, what voltage standard each pin uses, and how fast each clock runs. A board's constraints file is written once and reused by every design for that board.

These steps can take seconds for a small design and many minutes for a large one, because placement and routing are difficult search problems. They usually run on a larger computer, but for small designs they can run on the FPGA computer itself, which can then build new hardware for itself.

15.9 Reading the Reports

The tools report two numbers that matter on every build.

Utilization states how much of the chip the design uses: the LUTs, block RAMs and other resources used, out of those available. As a chip fills, the placer has less room to work with and timing becomes harder to meet. Above about three quarters full, designs become noticeably harder to build.

Timing gives the maximum frequency of each clock, based on the critical path described in Digital Logic. If it is below the frequency at which the design runs, the circuit may work on the bench and fail when warm, or fail in rare and confusing ways. A design that does not meet timing is not complete.

15.10 Classic Mistakes

- **Blocking assignments in clocked blocks.** Registers should be assigned with `<=`.

- **Accidental latches.** In a combinational `always @*` block, every output must be assigned in every case. If an `if` has no `else`, the tool must keep the old value and so builds a latch, usually unintentionally.
- **Two drivers.** A signal can be driven from only one place. Assigning the same register in two different `always` blocks fails in synthesis, even if the simulator accepts it.
- **Missing reset.** Registers that must start in a known state need a reset or an initial value.
- **Unsynchronized inputs.** A button or a signal from another clock must pass through two flip-flops before it is used, as described in Digital Logic.
- **Width mismatches.** Assigning a 32-bit value to an 8-bit register silently drops the top bits. Tools warn about this, and the warnings should be read.
- **Simulation-only code.** Delays such as `#1`, printed messages and the scripted steps of a testbench cannot become hardware, and synthesis ignores or rejects them. A design that depends on them works only in the simulator.
- **Making clocks with logic.** Dividing a clock with a counter and using the result as another clock causes timing problems. The alternatives are to use the PLL, or to keep one clock and use the counter as an enable.

The blink module above follows the last rule. It does not create a slow clock; it uses the top bit of a counter that runs on the fast one.

16 Systems on a Chip

A **system on a chip** (SOC) places a complete computer, with its processor, memory controller, graphics and peripherals, on a single chip. The processors in phones and tablets are SOCs: each is designed once, made in millions, and cannot be changed once made. In an FPGA, an SOC is a design like any other, often called **gateware** by analogy with software. Every part of it can be read and changed. This chapter describes how an SOC is organized, and follows one small addition from hardware to software.

16.1 Soft Processors

A processor described in an HDL and built in FPGA logic is a **soft processor**, or softcore. Some FPGAs also contain a fixed **hard processor** beside their logic, which is faster but cannot be changed.

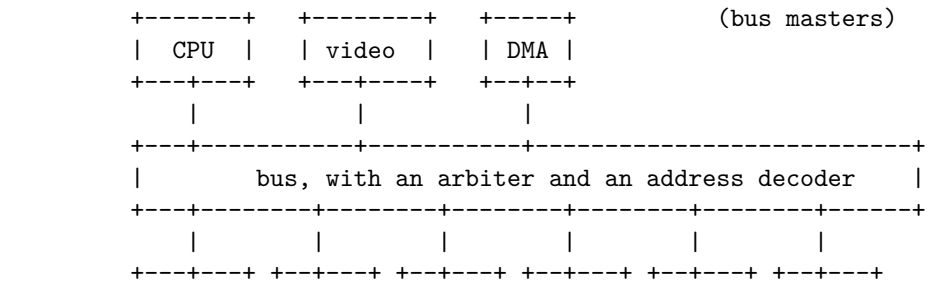
There are many open softcores for RISC-V. Some are small and simple, completing an instruction every few cycles and using one or two thousand LUTs. Others are pipelined and add caches and memory management, at several times the size. The Zeitlos SOC uses PicoRV32, a small, well-tested RISC-V core, and also has an experimental core of its own, Zeitlos32.

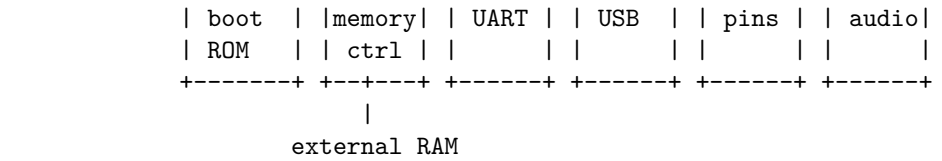
A soft processor runs much more slowly than a hard one, typically at tens of megahertz. Its advantages are:

- **It can be read.** Every gate of the processor is visible in its source, so it can be audited, studied and trusted to a degree that no commercial chip allows.
- **It can be fixed.** A processor bug is corrected by changing the source and rebuilding the bitstream.
- **It can be extended.** New instructions or features can be added for a particular need.
- **It can be replaced.** A machine built with one instruction set can later be given another, by loading a different design into the same hardware.

16.2 The Shape of an SOC

A typical FPGA computer is organized as follows:





- The **CPU** fetches instructions and moves data over the bus.
- Other **bus masters**, such as the video circuit that reads the framebuffer and the DMA engines described in The Computer, use the bus as well. An **arbiter** decides which master uses the bus when two request it at once.
- An **address decoder** turns each address into a select signal for one device.
- The **boot ROM** is a small program held in block RAM, built into the bitstream itself, so that the processor has a program to run as soon as the FPGA is configured. It loads the operating system from flash.
- The **memory controller** connects the bus to external SDRAM or DDR memory.
- The **peripherals** each present a few registers on the bus.

16.3 Adding a Device, End to End

This section traces how a program affects the physical world, using the smallest useful peripheral: one register that the processor can write and read back, whose bits drive eight LEDs.

```
// A peripheral with one register: a byte the CPU can write and read
// back, whose bits drive eight LEDs.
module led_reg (
    input  wire        clk,
    input  wire        reset,
    input  wire        sel,        // the bus is addressing this device
    input  wire        we,         // and it is a write
    input  wire [31:0] wdata,      // the data the CPU is writing
    output wire [31:0] rdata,      // the data the CPU reads
    output wire [7:0]  leds
);
    reg [7:0] value;

    always @(posedge clk)
        if (reset)
            value <= 8'd0;
        else if (sel && we)
            value <= wdata[7:0];

    assign rdata = {24'd0, value};
    assign leds  = value;
endmodule
```

The register `value` loads the bottom byte of the bus data at a clock edge when this device is selected and the bus is writing. Its contents can be read back at any time and always drive the `leds` output.

The device is selected by the address decoder. If it is to answer at `0xE000_0000`, the decoder needs one line:

```
// the address decoder: the top four bits pick the device
wire sel = (addr[31:28] == 4'hE);
```

The `leds` output is connected to eight pins of the FPGA, and the board's constraints file says which ones. After synthesis, place and route and packing, the new bitstream contains the device.

On the software side, the function `led_on` from the chapter C is a single line, `PINS |= 1u`, where `PINS` names the 32-bit value at address `0xE000_0000`.

A call to `led_on` proceeds as follows:

1. The compiler turns `PINS |= 1u` into an instruction that puts the address `0xE000_0000` in a register, then three more: a load from that address, an OR with 1, and a store back.
2. The load puts the address on the bus. The decoder sees `E` in the top four bits and selects `led_reg`, which places its register on `rdata`. The value comes back into a CPU register.
3. The ALU sets bit 0.
4. The store puts the address and the new value on the bus with write enable set. At the next clock edge, `value` loads the new byte.
5. The `leds` output changes, the pin goes to 3.3 V, current flows through the resistor and LED from Electricity, and the light comes on.

These five steps involve every layer of this book. Larger peripherals, such as a serial port, a sound mixer or a USB controller, work in the same way, with more registers and more logic behind them.

16.4 Hardware and Software Together

Because its hardware can be changed, an FPGA computer allows a choice that ordinary computers do not: whether a task is done in software or in hardware.

Software is easier to write and change. Hardware can be much faster for some tasks, because it can perform many operations at once and does not spend cycles fetching and decoding instructions. Suitable tasks are small, regular and frequently repeated:

- copying and filling rectangles of pixels, done by a **blitter**
- drawing lines
- mixing several streams of sound
- the multiplications of large numbers on which public-key cryptography depends

- hash functions such as SHA-256

The Zeitlos SOC can include hardware for each of these, and each makes practical on a 48 MHz processor a task that would otherwise be too slow. The usual approach is to write everything in software first, measure where the time is spent, and move only the slowest part into hardware, behind a few registers.

16.5 Moving a Design to Another Board

A design written in an HDL is not tied to one board, and moving it to another is the most common way of adapting an FPGA computer. Most of the design is unchanged: the processor, the bus, the peripherals, the address map and every program that runs on them. The changes concern the physical board:

- **Pins.** The constraints file is rewritten for the new board: which FPGA pin carries each signal, and with what voltage standard. Every pin must suit the voltage of its bank.
- **Clocks.** The new board's crystal may run at a different frequency, so the PLL settings change. A slower or fuller chip may not meet timing at the old clock speed, and the design may need to run slower.
- **Memory.** SRAM, SDRAM, DDR and PSRAM each need a different memory controller, with timings for the particular chip. The memory size changes what software can do.
- **Chip-specific blocks.** PLLs, block RAM, special I/O cells and differential outputs differ between FPGA families. A well-organized design keeps each of them in a small wrapper module, so that only the wrapper changes.
- **Capacity.** A smaller chip may not hold everything. Optional parts, such as accelerators or a second serial port, are left out with a configuration switch.
- **Outputs.** VGA, HDMI and composite video, USB and audio each depend on what the board has.

The usual practice is to keep everything board-specific in one place: a top-level module, a constraints file and a short list of configuration settings for each board. A new board is then brought up in small steps: first a blinking LED, to confirm the clock, the pins and the programming method; then a serial port, for messages; then a memory test; and finally the full system. One SOC design can thus support a whole family of boards, including boards from other makers.

16.6 Linux on an FPGA

Linux can also run on an FPGA computer. It needs a softcore with a memory management unit, as described in Operating Systems, and at least tens of megabytes of memory. LiteX, an open framework for building SOCs, can generate a Linux-capable system for many FPGA boards.

Kakao Linux, a small distribution for Machdyne's larger computers, is built this way. Linux provides thousands of existing programs, at the cost of size,

complexity and start-up time. A system designed for the hardware can be small enough to be understood completely. With the configuration flash on a removable module, the same computer can run either.

16.7 Recreating Old Machines

An FPGA can also be configured as a computer from the past. Projects have recreated, closely and sometimes down to individual gates, the processors and custom chips of the Apple II, the Commodore 64, the Amiga, early game consoles and many more, from documentation, from careful measurement and sometimes from microscope photographs of the original chips.

This differs from ordinary emulation, in which a program on a modern processor imitates the old one instruction by instruction. The FPGA becomes hardware that behaves like the original, clock cycle by clock cycle, so software that depended on precise timing also runs correctly. It is one of the most reliable ways to preserve old software, and shows that a computer's design can outlive the chips it was first made from, which is a central aim of timeless computing.

17 Small Computers

Not every part of a system needs hardware that can be redesigned. Most computers in the world have no screen and no keyboard. They are built into keyboards, chargers, washing machines, cars and storage devices, and each runs one program for its whole life. These are **microcontrollers**, and they are well suited to many timeless applications, especially automation. This chapter describes them, and explains how small programmable modules can be combined with a larger computer to build systems that last.

17.1 Microcontrollers

A **microcontroller** is a complete computer on one inexpensive chip, with a processor, flash memory for its program, a small amount of RAM and a set of built-in peripherals. Typical peripherals are:

- general-purpose pins
- timers, for measuring time and producing PWM signals
- analog-to-digital converters
- UART, SPI and I2C controllers

A small RISC-V microcontroller might have 16 KB of flash and 2 KB of RAM. This is enough for reading sensors, switching outputs and making decisions many thousands of times a second.

Unlike an FPGA computer, a microcontroller is fixed. Its processor and peripherals are set by the manufacturer; only the program can be changed. In return, a microcontroller is cheap, small, easy to use and economical with power. Many can sleep on a few microamps and wake on a timer or a pin change, so they can run for years from a small battery.

The program in a microcontroller is called its **firmware**. It is usually written in C, compiled on another computer, and written into the chip's flash through a programming interface: either a debug connection of one or two wires, or a small resident program called a **bootloader** that accepts new firmware over a serial port or USB.

17.2 An Interpreter on the Chip

Alternatively, an interpreter can be written into a microcontroller's firmware once, and programs sent to it afterwards as text. Machdyne's modules work this way, with Machdyne BASIC as their firmware.

This approach has several advantages for timeless computing:

- **No toolchain is needed** to change a program. Any computer, or any device that can send text over a serial line or a bus, can program the module.

- **Programs are readable.** The program inside the module is plain text that can be listed, read and changed in place years later, by someone who has never seen it before.
- **Programs are portable.** A BASIC program does not depend on the chip that runs it, so it can be moved to a different module when the old chip is no longer made.

The disadvantage is speed. An interpreted program runs many times more slowly than compiled firmware. For a controller that checks a temperature once a second, this does not matter.

17.3 Modules

A **module**, in this sense, is a microcontroller on a small circuit board, with enough around it to be useful on its own and a standard set of contacts for connecting it to a larger system.

Machdyne's modules are designed to last. Each combines a RISC-V microcontroller with long-lived non-volatile memory, either F-RAM or EEPROM, which holds its programs and data as files. The first of them, the LS10, has a microcontroller with 16 KB of flash and 2 KB of RAM, and 8 KB of F-RAM. The modules fit a 12-pin footprint that can be soldered to a board, pushed into a socket or plugged into a six-pin Pmod connector. The first six pins follow a standard called **Sechs**.

17.4 The Sechs Interface

Sechs, German for "six", defines six contacts:

Pin	Name	Use
1	A	shared I2C bus: clock
2	B	shared I2C bus: data
3	C	the module's own I/O
4	D	the module's own I/O
5	GND	ground
6	3V3	3.3 V power

Pins A and B form an I2C bus, as described in Talking to the World, which is shared by every module connected to it. Pins C and D belong to each module, for its sensor, its output, a local I2C device or a serial console.

Sechs describes three roles:

- a **module**, which runs programs
- a **carrier**, the board a module plugs into, which supplies power and determines the use of pins C and D

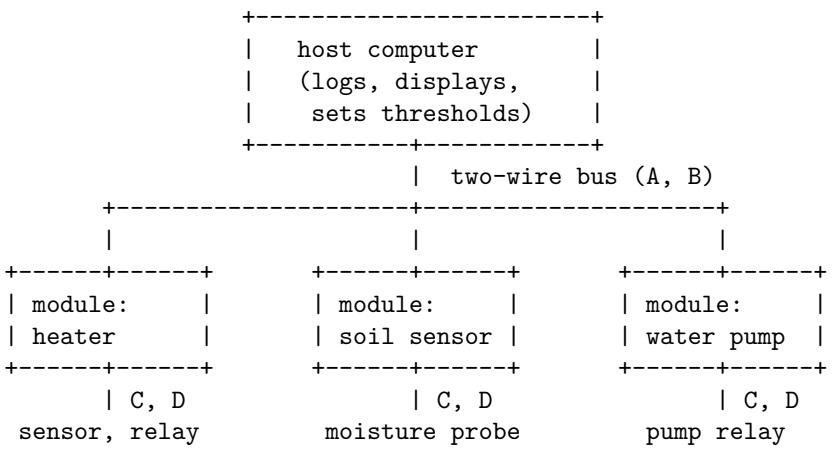
- a **master**, usually a host computer, which controls modules over the shared bus

Over the bus, a host can find every module by scanning addresses and ask each one what it is. It can stop and start a module's program, read and write its sixteen program registers, send it a new program, and open a console to type commands into it directly. A new module answers at a default address, and the host can assign each module its own address so that several can share a bus.

The safety rules described in Talking to the World make this robust. Sechs is still a draft standard, and the details of its registers may change, but these rules are its foundation. A module keeps its pins inactive at power-on. It always gives the host a short period after power-on in which to stop it, so that no program, however faulty, can make the module unreachable. A new program replaces the old one only after it has arrived complete and been checked.

17.5 Building Systems from Modules

Modules allow a larger system to be built by giving each task its own small computer.



In this greenhouse, each module runs its own `BOOT` program, like the heater controller in BASIC. Each module continues to work if the host is switched off or fails. The host reads each module's registers to log and display its state, and writes registers to change settings. A failed module can be replaced with a spare and reprogrammed in minutes, while the rest of the system continues to run.

This design follows the principles in Timeless Computing. Each part is simple enough to be understood completely, and each can be replaced. A failure affects one part rather than the whole system. The programs are stored as text in long-lived memory, so whoever maintains the system in future can read and change them.

17.6 Portability: Machines and Languages

There are two general ways to make programs outlive the hardware they were written for. The first is a common **instruction set**, or virtual machine: every device runs the same machine code, either directly or through a small interpreter. The second is a common **language**: every device accepts the same source programs and runs them in whatever way suits its chip.

Machdyne's modules have used both. Their predecessor, the Zwölf platform, defined a small 8-bit processor with a stack and a few instructions, which every module was to implement regardless of the chip it was built from. Sechs keeps the six-pin interface but does not define a common instruction set. Portability comes instead from Machdyne BASIC, which can be interpreted on some chips and could be compiled on others.

A common instruction set makes compiled programs portable, and many systems have used one, from early educational computers to modern bytecode languages. A common language is simpler to build, and leaves each device free to run programs in its own way. In either case, what matters is a stable, documented layer between the programs and the hardware, so that either can change without the other.

17.7 Module Lifetime

Two measures describe the life of a small device. **Data retention** is how long its memory keeps what was written to it. For F-RAM, as described in Memory and Storage, it is rated in centuries. The **functional life** of the device as a whole depends on its other parts: the microcontroller, the solder joints, the connectors, the capacitors and its operating conditions, and it is generally shorter.

Modules address this in the same way as the rest of the book. The important information is kept in the most durable form, the parts are cheap and replaceable, and the design and documentation are open, so that a replacement can always be made.

The next part combines this material in one complete machine.

Part VI

The Machine

18 The Machine

The previous chapters described computers in general. This chapter describes a particular computer: a Machdyne FPGA computer running Zeitlos. It does not assume that the reader has one. It explains how the ideas of this book appear in a real machine, how it can be adapted, and how it can be returned to a working state when an experiment goes wrong.

Zeitlos changes quickly, and the system's own documentation describes it in exact detail. This chapter gives an overview that makes that documentation easier to read.

18.1 What It Is

With a keyboard, mouse, monitor and USB power supply connected, the machine starts to a graphical desktop. The screen is 640 by 480 pixels in a single colour, in the manner of early workstations: white, amber or green on black, or black on white like paper. A dock along the bottom starts programs, and windows can be moved, resized and stacked.

The programs supplied with it cover the timeless applications listed in Timeless Computing: a text editor, a file browser, a document reader, a spreadsheet, a calculator, a calendar and clock, a terminal, an image viewer, music and sound players, a web browser for simple pages, Gopher and Gemini, and a chat client, along with games and educational programs. This book is among the documents it carries.

The core of the system, consisting of the kernel, the drivers, the window manager, the terminal, the editor, the file browser and the Scheme language, is stored in the computer's flash memory, so the machine works with no storage card at all. A microSD card adds further programs and space for files.

18.2 The Hardware

Machdyne makes several FPGA computers, and Zeitlos also runs on some boards from other manufacturers, so details vary. The common parts are:

- an **FPGA**, usually a Lattice ECP5, which becomes the whole computer
- **external RAM**, from 1 MB of SRAM to tens of megabytes of SDRAM, with larger DDR3 memory on boards still being brought up
- **configuration flash**, often on a removable MMOD module, holding the FPGA's design and the core of the operating system
- **USB host ports** for a keyboard, a mouse, storage sticks and network adapters
- **video output**: DVI through an HDMI connector, VGA, or composite for a television
- a **microSD** slot
- **Pmod** sockets for expansion

- power from a USB supply, a few watts

18.3 The System on the Chip

Each part of the Zeitlos SOC was described in an earlier chapter:

In this book	In the Zeitlos SOC
The Computer	RISC-V CPU (RV32IM), 48 MHz, PicoRV32 core
The Computer	Wishbone bus, address map, caches
Operating Systems	address translation per program, memory protection
Talking to the World	framebuffer, blitter and line drawing
Talking to the World	8-channel audio mixer, USB host, serial ports
Talking to the World	pins on the Pmod sockets, I2C and SPI
Communication	Ethernet, random number generator
Systems on a Chip	multiplier and hash accelerators for cryptography

One address space, translated per program. There is no full memory management unit. Each program is loaded somewhere in main memory, but always sees itself at the same address, `0x8000_0000`; the hardware adds an offset when it switches programs. Programs therefore need no adjustment for where they are loaded, and the kernel can pass data between programs by passing addresses.

Protection for containment, not secrecy. A memory protection unit prevents a program from writing outside its own memory or to devices it is not permitted to use. A faulty program is stopped with a report, and everything else continues. Zeitlos is a single-user system, and this protection exists to contain errors, not to defend against a hostile program the owner chose to run.

There is also a **game mode**, in which a 320 by 240 region of the screen is doubled in size to fill the display and, on boards built with colour, can use up to sixteen colours. It is used by games and by the BASIC computer described below.

18.4 Starting Up

The start-up sequence closely follows Operating Systems:

1. The FPGA loads the Zeitlos design from its configuration flash and becomes the computer. Some boards first run a small bootloader that waits a few seconds for an update over USB.
2. A boot program, built into the design in block RAM, copies the kernel from flash into main memory and starts it.
3. The kernel starts the filesystem driver, then the USB driver, reads the configuration file, and starts the window manager, the network driver and any other services the configuration specifies.

4. The desktop appears.

A program is looked for first in the `/apps` folder of the SD card, then in flash; only the kernel and the filesystem driver always come from flash. As a result, a newer version of a program can be tried by copying it onto the card, and the original is restored as soon as the copy is deleted.

18.5 The Operating System

Zeitlos is a **microkernel**, as described in Operating Systems. The kernel schedules programs, manages memory, passes messages, routes interrupts and enforces protection. The filesystem, USB and network are driver programs that run like any other. If one crashes, the kernel restarts it a moment later, and most programs using it recover, although open files may need to be opened again.

Scheduling is preemptive, driven by a timer several hundred times a second. When a program sends a message and waits for the answer, the processor switches directly to the program that will answer it. As a result, the desktop responds immediately on a 48 MHz processor: work passes directly to where it is needed, and programs waiting for an event use no time.

Programs communicate through messages and through **ports**: named services that other programs can connect to. A terminal window can connect in the same way to the Scheme interpreter, to a shell modelled on Unix (following POSIX, the standard that describes Unix-like systems), to the BASIC computer, to a serial line, or to another machine across the network.

Files are arranged in one tree:

Path	Contents
/	the SD card, with the flash underneath
/apps	programs
/data	each program's own files, one folder each
/docs	documentation, including this book
/sys	the system configuration
/ram	a RAM disk, fast and lost at power-off
/usb	a USB stick, when one is plugged in
/home	the owner's files

Settings are kept in a plain text configuration file, which can be read and edited like any other. A few settings that belong to the machine rather than to a card, such as the password, are kept in a small, power-safe area of flash.

18.6 The Languages

All three languages from the programming chapters are on the machine.

Scheme is always available, from flash, through a terminal window. Besides standard Scheme, it has procedures for files, windows and drawing, starting and stopping programs, simple network transfers, and the pins and buses on the expansion sockets, each with built-in help. It is the quickest way to automate a task or to explore the system.

BASIC runs as a separate small computer, in the tradition of the 1980s. It has a 320 by 240 screen, shown in a window or full screen, graphics statements and its own files in one folder, but no access to the rest of the system, which makes it a safe environment for learning. Programs for Machdyne's modules can be written and tried here, or on a virtual module that imitates the limits of a real one exactly, before being sent to a real module.

C programs are usually compiled on another computer with the standard open tools, as Zeitlos itself is. An experimental compiler that runs on the machine itself is being brought up. On machines with enough memory, the Unix-style shell provides familiar commands, pipes and the vi editor.

18.7 Adapting the Machine

The machine can be adapted at several levels, each deeper than the one before.

1. Settings. Screen mode and colour, keyboard layout, time zone, the services that start, and many other settings are lines in the configuration file. Most can be changed with the settings program.

2. Scripts and schedules. A few lines of Scheme can rename files, process text, fetch data from the network or open a small window with buttons. A built-in scheduler runs programs at given times or intervals, or at start-up.

3. Programs. A program written in C, compiled and placed in `/apps` becomes an application like the others, with its own window if it needs one.

4. Hardware. The Pmod sockets give direct access to pins from Scheme and C, with I2C and SPI built on top. Sensors, displays and relays connect here, as do Sechs modules: a command in the Unix-style shell can find modules on the bus, read their registers and send them BASIC programs. A virtual workbench can simulate parts such as pin expanders, LEDs and buttons on screen, so that a program's logic can be tested before anything is wired.

5. The gateware. The SOC itself is open Verilog. With the open FPGA tools on another computer, it can be modified, extended with a peripheral as described in Systems on a Chip, and rebuilt. For small designs, an experimental set of tools on the machine can synthesize, place and route a design and generate its bitstream, without any other computer.

6. The system. The source of every program and of the kernel is available. Any program can be rebuilt and tried by placing the new version in the card's `/apps` folder while the original remains in flash. The kernel, the filesystem driver

and the gateware are changed by rewriting the flash, which is best done on a spare MMOD.

18.8 Getting Back to Safety

The machine is designed so that a failed experiment can always be undone:

- **The core system is in flash**, and the machine starts without a card. A card with a faulty program on it can be removed.
- **Card copies override flash copies** for programs. Deleting the copy in `/apps` restores the original.
- **A rescue console** appears if the desktop cannot start, and can be called up by holding Escape during start-up. From it, programs can be started by hand and problems investigated.
- **Crashes are contained.** A faulty program is stopped and reported; a faulty driver is restarted, up to a few times a minute.
- **Trial gateware is not kept.** A design sent from another computer for a trial lasts only until the power is turned off. The region of flash holding the bootloader is protected from any action Zeitlos can take.
- **A second MMOD** is the safest place to try a new design. It can be fitted for an experiment and the original refitted afterwards. The machine can also copy its own configuration module to a spare as a backup.
- **Release images** restore the whole machine to its original state, and are written to the flash with the open tools from another computer.

With a spare MMOD holding a known good system, and a copy of the release image, almost any change to the machine can be undone in a few minutes.

18.9 Where to Go Next

The system's own documentation is in the `/docs` folder and can be read in the document reader. It begins with an index and a welcome page that covers the desktop and every keyboard shortcut, and has a page for most programs, each language and each part of the hardware. In the Scheme interpreter, `help` followed by a name explains any procedure.

Zeitlos is under active development. Some features described in its documentation are marked experimental, and some boards are better supported than others. Its documentation states which, and is always more current than a book.

19 Computing That Lasts

Most people assume that electricity, spare parts, software updates and the Internet will always be available. Usually they are, but they are not guaranteed, and in some parts of the world they are already scarce. Natural disasters, conflicts, economic difficulties, changes in law and the decisions of distant companies can each remove something a computer depends on.

Survival computing considers how computing could continue under those conditions. It is best treated as a design exercise rather than a prediction. The properties that make a computer more likely to survive difficult times also make it cheaper, simpler and easier to own in ordinary times. This chapter collects the practical lessons of the whole book.

19.1 Power Supply

Know how much power the equipment uses, and therefore how much energy it needs. Energy is power multiplied by time, measured in **watt-hours** (Wh). A computer drawing 3 W for eight hours a day needs 24 Wh a day.

This is a small amount. A common USB power bank stores 40 to 100 Wh, enough for one to four days of such use. A small solar panel can replace the energy used. A 20 W panel might deliver about 50 to 100 Wh on a sunny day, depending on season, latitude and weather, and much less on an overcast one. Allow a generous margin, and enough battery capacity for several cloudy days.

Several practices make the energy last longer:

- **The display often uses the most power.** A small monitor, or a television on composite video, may draw more than the computer.
- **Unused equipment can be switched off.** A module that sleeps between readings uses almost no energy.
- **A slower clock uses less power.** CMOS circuits use power mainly when they switch, as described in Electricity.
- **Conversion wastes energy.** Each step from one voltage to another loses some, so equipment that runs directly from 5 V USB power wastes less.

19.2 The Environment

Electronic components age at a rate determined by their conditions.

Heat is the most important cause of ageing. It accelerates nearly every kind of wear: memory loses data sooner, as described in Memory and Storage, capacitors dry out, and batteries degrade. A rough rule for many components is that every additional ten degrees Celsius halves their life. Keep equipment and stored media cool.

Moisture corrodes contacts and boards. Store equipment dry, and allow equipment brought in from the cold to warm up before switching it on, so that con-

densation can evaporate.

Dust blocks ventilation and can conduct when damp. **Static electricity** destroys chips, as described in Electricity. An enclosure protects against all of these. A case made with a 3D printer can also be replaced or modified freely.

19.3 Spares and Parts

A failed part cannot be replaced if no replacement can be found, so replacements should be planned in advance:

- Keep **spares of parts that wear out or are easily lost**: storage cards, configuration modules, cables, power supplies and small programmable modules.
- Prefer **standard connectors and voltages**, such as USB power, Pmod sockets and common batteries, which can be replaced with whatever is available.
- Prefer **socketed and modular parts**. A memory module that plugs in can be replaced by hand; one soldered to the board cannot.
- Keep **basic tools**: a multimeter, a soldering iron, small screwdrivers and a magnifier. With these and the schematics, most faults in a small computer can be found, and many repaired.
- Keep **the schematics and documentation** for every board in use, in a form that can be read without the board working.

19.4 Data

The rules from Memory and Storage can be restated as practices:

- Keep **at least three copies** of anything important, on more than one kind of medium, with one copy in another place.
- Use **plain formats**: text for writing, and simple, widely documented formats for everything else.
- Record a **hash** of each important file, and check copies against it periodically.
- **Refresh** copies onto new media before the old media reach the end of their expected life. Flash cards and USB sticks are for moving data, not for keeping it for decades.
- Keep **small, vital data**, such as keys, passwords and essential notes, on media built for longevity, such as F-RAM, and on paper.
- Keep **a list** of what is stored where, so that every copy can be found.

19.5 Software and the Means to Rebuild It

Software does not wear out, but it can become unusable. The programs that build it stop running on new computers, the websites that hosted it disappear, and the knowledge of how it was put together is lost.

The protection against this is to keep three things together, offline, and in more than one place:

- **the source code** of every program and design in use
- **the tools** needed to build it, ideally as source code themselves, together with the versions known to work
- **written instructions** for building and restoring everything, tested by following them

The most independent arrangement is a machine that can **rebuild itself**: one that can edit and compile its own software, and design and build its own hardware configuration, with no other computer involved. Each step towards this removes a dependency on something that might later be unavailable. Some small systems, including Zeitlos, work towards this with compilers and FPGA tools small enough to run on the machine they build for.

Recoverability is equally important: a known good state that can be restored in minutes. Keep a spare copy of the configuration flash and of the system image, and practise restoring them before they are needed.

19.6 Keeping Information

A computer without information has limited use. Collections of reference works, manuals, maps and books can be kept offline and read on small machines, especially in plain text. Such a collection should contain what would be needed if it could not be searched for: technical documentation for the equipment in use, references for the owner's work, and the knowledge the owner would want to pass on. It should be updated whenever possible.

19.7 People

The most durable store of knowledge is people. A machine that only one person understands depends on that person. Knowledge should therefore be taught, written down and shared within a community of people who use and maintain the same kinds of systems. Open designs make this possible: anyone can study them, and no permission is needed to repair, copy or improve them.

This book is intended to contribute to that work, and anyone may copy, print, translate and correct it.

19.8 A Checklist

- Know how much power the equipment needs, and have a way to supply it without the grid.
- Keep equipment cool, dry, clean and in an enclosure.
- Keep spares of the parts that wear out, and the tools and schematics to repair the rest.
- Keep three copies of important data, in plain formats, and check them.

- Keep the source, the tools and the instructions needed to rebuild the system, and test them.
- Keep a known good system image and a spare copy of the configuration flash.
- Keep an offline library of the information the work depends on.
- Learn how the machines work, and teach others.

Part VII

Appendices

A Glossary

Terms are explained in more detail in the chapter named in brackets.

- **ADC**: analog-to-digital converter; measures a voltage as a number. (Talking to the World)
- **address**: the number that identifies a location in memory or a device register. (The Computer)
- **address map**: the list of which device answers at which addresses. (The Computer)
- **algorithm**: a method for solving a problem in a finite number of exact steps. (Thinking in Programs)
- **ALU**: arithmetic logic unit; the part of a processor that calculates. (Digital Logic)
- **ASCII**: the 1963 standard that gives numbers to English letters, digits and symbols. (Information)
- **ASIC**: application-specific integrated circuit; a chip made for one design. (Inside an FPGA)
- **assembly language**: machine instructions written as readable text, one per line. (The Computer)
- **bare metal**: running a program directly on hardware, with no operating system. (Operating Systems)
- **baud rate**: the agreed speed of a serial line, in bits per second. (Talking to the World)
- **bit**: a single binary digit, 0 or 1. (Information)
- **bitstream**: the file that configures an FPGA. (Inside an FPGA)
- **blitter**: a circuit that copies and fills blocks of pixels without the processor. (The Computer)
- **block RAM**: dedicated memory blocks inside an FPGA. (Inside an FPGA)
- **bootloader**: the first program a computer runs, which loads the next. (Operating Systems)
- **bus**: the shared wires and rules that connect a processor to memory and devices. (The Computer)
- **byte**: eight bits. (Information)
- **cache**: a small, fast memory holding copies of recently used data. (The Computer)
- **carry chain**: a fast dedicated path for carries between FPGA logic blocks. (Inside an FPGA)
- **checksum**: a value computed from data, used to detect damage. (Memory and Storage)
- **clock**: a steady signal whose edges time a synchronous circuit. (Digital Logic)
- **closure**: a function together with the variables it captured where it was made. (Scheme)
- **CMOS**: the circuit style, using pairs of complementary transistors, of

nearly all digital chips. (Electricity)

- **combinational logic**: logic whose outputs depend only on its present inputs. (Digital Logic)
- **compiler**: a program that translates source code into machine code ahead of time. (Thinking in Programs)
- **constraints file**: the file that assigns a design's signals to FPGA pins and states its clock speeds. (Describing Hardware)
- **context switch**: saving one process's state and restoring another's. (Operating Systems)
- **CPU**: central processing unit; the processor. (The Computer)
- **CRC**: cyclic redundancy check; a strong kind of checksum. (Memory and Storage)
- **critical path**: the slowest path through a circuit, which limits its clock speed. (Digital Logic)
- **current**: the flow of electric charge, measured in amperes. (Electricity)
- **DAC**: digital-to-analog converter; produces a voltage from a number. (Talking to the World)
- **debouncing**: ignoring the rapid flicker of a switch as it opens or closes. (Talking to the World)
- **decoder**: a circuit that turns a binary number into one active output. (Digital Logic)
- **DMA**: direct memory access; a device reading or writing memory without the processor. (The Computer)
- **DRAM**: dynamic RAM; dense memory that must be refreshed constantly. (Memory and Storage)
- **driver**: software that operates one kind of device. (Operating Systems)
- **DSP block**: a dedicated multiplier inside an FPGA. (Inside an FPGA)
- **EEPROM**: electrically erasable memory, rewritten a byte at a time. (Memory and Storage)
- **F-RAM**: ferroelectric RAM; non-volatile, byte-writable, extremely durable memory. (Memory and Storage)
- **filesystem**: the rules that turn storage blocks into named files and folders. (Memory and Storage)
- **firmware**: the program built into a device such as a microcontroller. (Small Computers)
- **flash**: non-volatile memory erased in blocks; it wears with use. (Memory and Storage)
- **flip-flop**: a one-bit memory that changes only at a clock edge. (Digital Logic)
- **FPGA**: field-programmable gate array; a chip whose logic is set by a configuration file. (Inside an FPGA)
- **framebuffer**: the memory holding the image shown on the screen. (Talking to the World)
- **function**: a named, reusable piece of a program. (Thinking in Programs)
- **garbage collector**: the part of a language system that reclaims unused memory automatically. (Scheme)

- **gate**: a circuit computing a simple logical function such as AND or OR. (Digital Logic)
- **gateware**: the design loaded into an FPGA, as software is loaded into a processor. (Systems on a Chip)
- **GPIO**: general-purpose input/output; pins controlled directly by software. (Talking to the World)
- **ground**: the reference point of a circuit, defined as 0 volts. (Electricity)
- **hash**: a fingerprint of data; a cryptographic hash cannot practically be forged. (Memory and Storage)
- **heap**: memory requested and released by a program while it runs. (C)
- **hexadecimal**: base 16, written with the digits 0 to 9 and A to F. (Information)
- **HDL**: hardware description language, such as Verilog or VHDL. (Describing Hardware)
- **host**: the computer in charge of a bus or of attached devices. (Talking to the World, Small Computers)
- **I2C**: a two-wire bus connecting many devices by address. (Talking to the World)
- **interpreter**: a program that carries out another program directly, without translating it into machine code. (Thinking in Programs)
- **interrupt**: a signal that makes the processor stop and handle an event. (The Computer)
- **ISA**: instruction set architecture; the instructions a processor understands. (The Computer)
- **kernel**: the core of an operating system. (Operating Systems)
- **latch**: a memory element made from gates with feedback. (Digital Logic)
- **LED**: light-emitting diode. (Electricity)
- **little-endian**: storing the least significant byte of a word at the lowest address. (The Computer)
- **LUT**: look-up table; the small memory that implements logic in an FPGA. (Inside an FPGA)
- **memory-mapped I/O**: reaching devices by reading and writing addresses. (The Computer)
- **metastability**: a flip-flop caught between 0 and 1 by an input changing at the clock edge. (Digital Logic)
- **microcontroller**: a complete small computer on one chip. (Small Computers)
- **microkernel**: a kernel that runs drivers and services as separate processes. (Operating Systems)
- **MMOD**: a small socketed SPI memory module used in Machdyne computers. (Timeless Computing, Talking to the World)
- **MMU**: memory management unit; translates the addresses each program uses. (Operating Systems)
- **module**: in hardware, a small plug-in board; in Verilog, a unit of design. (Small Computers, Describing Hardware)
- **MPU**: memory protection unit; limits which memory a program may use.

(Operating Systems)

- **multiplexer**: a circuit that chooses one of several inputs. (Digital Logic)
- **netlist**: a list of logic elements and the connections between them. (Describing Hardware)
- **non-volatile**: keeping data without power. (Memory and Storage)
- **open-drain**: an output that can only pull a wire low. (Electricity)
- **overflow**: a result too large for the bits that must hold it. (Information)
- **packet**: a block of data with a header, sent across a network. (Communication)
- **PC**: program counter; the register holding the address of the next instruction. (The Computer)
- **PLL**: phase-locked loop; makes new clock frequencies from an existing clock. (Inside an FPGA)
- **Pmod**: a common six- or twelve-pin expansion connector. (Talking to the World)
- **pointer**: a variable holding a memory address. (C)
- **POSIX**: the standard that describes Unix-like operating systems. (The Machine)
- **port**: in networking, a number identifying a program on a machine; in Zeitlos, a named service. (Communication, The Machine)
- **process**: a running program. (Operating Systems)
- **protocol**: an agreement on the meaning and order of messages. (Talking to the World)
- **pull-up resistor**: a resistor that holds a wire high when nothing drives it. (Electricity)
- **PWM**: pulse-width modulation; varying the on-time of a fast pulse to control power. (Talking to the World)
- **RAM**: random-access memory, in which any address can be reached as quickly as any other; usually volatile working memory. (Memory and Storage)
- **recursion**: a function that calls itself. (Thinking in Programs)
- **register**: a group of flip-flops holding a word; also a device's addressable control location. (Digital Logic, The Computer)
- **REPL**: read, evaluate, print loop; an interactive language prompt. (Operating Systems)
- **RISC-V**: an open instruction set architecture. (The Computer)
- **scheduler**: the part of an operating system that decides which process runs. (Operating Systems)
- **Sechs**: a six-contact standard for small programmable modules. (Small Computers)
- **sequential logic**: logic with memory, whose outputs depend on past inputs. (Digital Logic)
- **shift register**: a chain of flip-flops that moves bits along at each clock edge. (Digital Logic)
- **SOC**: system on a chip; a complete computer on one chip. (Systems on a Chip)

- **softcore**: a processor built from FPGA logic. (Systems on a Chip)
- **SPI**: a fast four-wire serial bus. (Talking to the World)
- **SRAM**: static RAM; fast memory that needs no refresh. (Memory and Storage)
- **stack**: memory used last-in, first-out, holding each function call's data. (The Computer)
- **state machine**: a register holding a state, and logic that chooses the next state. (Digital Logic)
- **synthesis**: turning an HDL description into a netlist. (Describing Hardware)
- **system call**: a request from a program to the kernel. (Operating Systems)
- **tail call**: a call that is the last action of a function, which needs no new stack space. (Scheme)
- **TCP**: the protocol that provides reliable, ordered byte streams over IP. (Communication)
- **testbench**: a simulation-only module that tests a hardware design. (Describing Hardware)
- **transistor**: an electrically controlled switch. (Electricity)
- **truth table**: a table listing a logic function's output for every input combination. (Digital Logic)
- **two's complement**: the standard way of representing negative binary numbers. (Information)
- **UART**: a device that sends and receives bytes on a serial line. (Talking to the World)
- **USB**: universal serial bus. (Talking to the World)
- **UTF-8**: the standard encoding of Unicode text. (Information)
- **volatile**: losing data without power; in C, a value that may change outside the program's control. (Memory and Storage, C)
- **voltage**: the electrical pressure between two points, measured in volts. (Electricity)
- **word**: the natural unit of data for a processor, 32 bits in this book. (Information)
- **Zeitlos**: an open system on a chip and operating system for FPGA computers. (The Machine)

B Tables

B.1 ASCII Characters

The ASCII characters from 32 to 127, with their codes in decimal and hexadecimal. Codes 32 to 126 are printable; 127, DEL, is a control character. Capital and lowercase letters differ by exactly 32 (hex 20), a single bit, and the digit characters 0 to 9 are 48 to 57, so subtracting 48 from a digit's code gives its value.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
32	20	space	56	38	8	80	50	P	104	68	h
33	21	!	57	39	9	81	51	Q	105	69	i
34	22	"	58	3A	:	82	52	R	106	6A	j
35	23	#	59	3B	;	83	53	S	107	6B	k
36	24	$	60	3C	<	84	54	T	108	6C	l
37	25	%	61	3D	=	85	55	U	109	6D	m
38	26	&	62	3E	>	86	56	V	110	6E	n
39	27	'	63	3F	?	87	57	W	111	6F	o
40	28	(	64	40	@	88	58	X	112	70	p
41	29	)	65	41	A	89	59	Y	113	71	q
42	2A	*	66	42	B	90	5A	Z	114	72	r
43	2B	+	67	43	C	91	5B	[	115	73	s
44	2C	,	68	44	D	92	5C	\	116	74	t
45	2D	-	69	45	E	93	5D	]	117	75	u
46	2E	.	70	46	F	94	5E	^	118	76	v
47	2F	/	71	47	G	95	5F	_	119	77	w
48	30	0	72	48	H	96	60	`	120	78	x
49	31	1	73	49	I	97	61	a	121	79	y
50	32	2	74	4A	J	98	62	b	122	7A	z
51	33	3	75	4B	K	99	63	c	123	7B	{
52	34	4	76	4C	L	100	64	d	124	7C	|
53	35	5	77	4D	M	101	65	e	125	7D	}
54	36	6	78	4E	N	102	66	f	126	7E	~
55	37	7	79	4F	O	103	67	g	127	7F	DEL

B.2 Control Characters

The codes below 32 control a terminal or printer instead of printing. These are the ones still in common use:

Dec	Hex	Name	Keyboard	Use
0	00	NUL		ends a string in C
3	03	ETX	Ctrl+C	interrupt a running program
4	04	EOT	Ctrl+D	end of input
7	07	BEL	Ctrl+G	sound the bell

Dec	Hex	Name	Keyboard	Use
8	08	BS	Ctrl+H	backspace
9	09	HT	Ctrl+I, Tab	horizontal tab
10	0A	LF	Ctrl+J	line feed: new line
13	0D	CR	Ctrl+M, Enter	carriage return
27	1B	ESC	Esc	starts terminal escape sequences

B.3 Powers of Two

n	2 to the n	Common name
4	16	values in a nibble
8	256	values in a byte
10	1,024	1 KiB
12	4,096	a common flash sector
16	65,536	64 KiB; 16-bit values
20	1,048,576	1 MiB
24	16,777,216	16 MiB; 24-bit colours
30	1,073,741,824	1 GiB
32	4,294,967,296	4 GiB; 32-bit values

B.4 Unit Prefixes

Prefix	Symbol	Factor	Example
giga	G	1,000,000,000	1 GHz processor clock
mega	M	1,000,000	48 MHz
kilo	k	1,000	10k resistor
milli	m	0.001	5 mA through an LED
micro	u	0.000001	10 uA in sleep
nano	n	0.000000001	21 ns per clock cycle at 48 MHz

The proper symbol for micro is the Greek letter mu; a plain u is the usual substitute where it cannot be typed.

B.5 Electrical Formulas

```
V = I x R              Ohm's law: volts = amps x ohms
P = V x I              power: watts = volts x amps
E = P x t              energy: watt-hours = watts x hours
R = (Vsupply - Vled) / I     series resistor for an LED
f = 1 / T              frequency = 1 / period
```

C A Short History

Computing builds on more than two thousand years of discovery. This timeline lists some of the steps relevant to this book.

C.1 Discovering Electricity

- **About 600 BC.** Thales of Miletus is said to have noticed that rubbed amber attracts light objects. The Greek word for amber, *elektron*, gives electricity its name.
- **1600.** William Gilbert publishes a study of magnetism and static electricity, and coins the Latin word *electricus*.
- **1780s.** Luigi Galvani observes that electricity makes a frog's leg twitch.
- **1800.** Alessandro Volta builds the first battery, the voltaic pile, giving experimenters a steady current for the first time.
- **1820.** Hans Christian Oersted discovers that an electric current deflects a compass needle, linking electricity and magnetism.
- **1827.** Georg Ohm publishes the law relating voltage, current and resistance.
- **1831.** Michael Faraday discovers electromagnetic induction, the principle of the generator and the transformer.
- **1830s and 1840s.** The electric telegraph, including Samuel Morse's system and code, sends messages across countries at the speed of electricity.
- **1860s.** James Clerk Maxwell's equations unify electricity, magnetism and light.
- **1876.** Alexander Graham Bell patents the telephone.
- **1880s.** Electric power networks are built. Alternating current, developed by Nikola Tesla and others, wins out for long-distance transmission.
- **1897.** J. J. Thomson identifies the electron.

C.2 Electronics

- **1904.** John Ambrose Fleming invents the vacuum tube diode.
- **1906.** Lee de Forest adds a control grid, making the triode, which can amplify and switch signals.
- **1947.** John Bardeen and Walter Brattain, in William Shockley's group at Bell Labs, build the first transistor.
- **1958 and 1959.** Jack Kilby at Texas Instruments and Robert Noyce at Fairchild Semiconductor independently invent the integrated circuit.
- **1963.** CMOS logic is invented at Fairchild.
- **1965.** Gordon Moore observes that the number of transistors on a chip is doubling at a steady rate, a trend that held for decades.
- **1985.** Xilinx sells the first commercial FPGA.

C.3 Computers

- **1837.** Charles Babbage designs the Analytical Engine, a mechanical general-purpose computer that was never completed.
- **1843.** Ada Lovelace publishes notes on the Analytical Engine, including a method for computing Bernoulli numbers on it, often called the first computer program.
- **1854.** George Boole publishes the algebra of logic.
- **1936.** Alan Turing describes a theoretical machine that can carry out any computation, laying the foundation of computer science.
- **1937.** Claude Shannon shows that Boole's algebra can describe and design switching circuits.
- **1941.** Konrad Zuse completes the Z3, an electromechanical programmable computer.
- **1944.** Colossus, designed by Tommy Flowers, breaks codes at Bletchley Park, the first programmable electronic digital computer.
- **1945.** ENIAC, built by John Mauchly and J. Presper Eckert, is completed. It is reprogrammed by rewiring.
- **1948.** The Manchester Baby runs the first program stored in electronic memory.
- **1958.** John McCarthy creates Lisp.
- **1964.** John Kemeny and Thomas Kurtz create BASIC at Dartmouth College.
- **1969.** The ARPANET, ancestor of the Internet, links its first computers. Work on Unix begins at Bell Labs.
- **1971.** Intel's 4004 puts a whole processor on one chip.
- **1972.** Dennis Ritchie creates C.
- **1975.** Guy Steele and Gerald Sussman create Scheme. The Altair 8800 kit starts the personal computer era.
- **1977 to 1985.** Home and personal computers spread: the Apple II, the TRS-80, the Commodore 64, the BBC Micro and the ZX Spectrum, all of which started up into BASIC, followed by the IBM PC, the Macintosh and the Amiga.
- **1978.** The first public bulletin board system goes online.
- **1983.** The ARPANET switches to TCP/IP.
- **1984.** Verilog is created.
- **1989 to 1991.** Tim Berners-Lee creates the World Wide Web.
- **1991.** Linus Torvalds releases Linux.

C.4 Open Hardware

- **2010.** Work on RISC-V begins at the University of California, Berkeley. Its specification is published openly and anyone may implement it.
- **2015.** Project IceStorm documents the configuration format of the Lattice iCE40 and releases the first complete open-source FPGA toolchain. Open tools for other FPGA families follow.

D Three Languages Side by Side

Euclid's algorithm from Thinking in Programs, finding the greatest common divisor of 1071 and 462, written in each of the three languages. Each prints or returns 21.

BASIC, with a loop built from `IF` and `GOTO`:

```
10 A = 1071: B = 462
20 IF B = 0 THEN GOTO 60
30 T = A MOD B
40 A = B: B = T
50 GOTO 20
60 PRINT A
```

Scheme, as a recursive function. The recursive call is a tail call, so this runs as a loop:

```
(define (gcd-of a b)
  (if (= b 0)
      a
      (gcd-of b (remainder a b))))

(gcd-of 1071 462)              ; => 21
```

C, with a `while` loop:

```
int gcd(int a, int b)
{
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

D.1 Common Constructs

Construct	BASIC	Scheme	C
comment	REM note	; note	/* note */
assign	A = 5	(set! a 5)	a = 5;
arithmetic	A * (B + 1)	(* a (+ b 1))	a * (b + 1)
remainder	A MOD B	(remainder a b)	a % b
equality	A = B	(= a b)	a == b
choice	IF A > 0 THEN …	(if (> a 0) x y)	if (a > 0) …
loop	FOR I = 1 TO 10	(let loop ((i 1)) …)	for (i = 1; …)

Construct	BASIC	Scheme	C
function	GOSUB 100	(define (f x) …)	int f(int x)
output	PRINT A	(display a)	printf(“%d”, a);
truth	-1, 0	#t, #f	non-zero, 0

E Sources

This book does not assume access to a network. For readers who have one, this appendix lists where to find the projects it describes, their source code and documentation, and further reading. Web addresses change over time; if one no longer works, search for the project's name.

E.1 Machdyne

What	Where
Machdyne	https://machdyne.com
Zeitlos	https://zeitlos.org
Community bulletin board	telnet bbs.machdyne.com
Source code of all projects	https://github.com/machdyne

The source of each project mentioned in this book, under https://github.com/machdyne:

Repository	Contents
tc	this book, with the tests for its examples
zeitlos	the Zeitlos SOC and operating system
basic	Machdyne BASIC, its language reference and module firmware
ms	Machdyne Scheme
sechs	the Sechs module interface standard
zwolf	the Zwölf module platform, predecessor of Sechs
mmod	the MMOD memory module standard
kakao	Kakao Linux for Machdyne computers
ark	an offline information collection
blaustahl	the Blaustahl F-RAM storage device
kaltstahl	the Kaltstahl F-RAM storage device
werkzeug	the Werkzeug multi-tool
zucker	Zucker, the SOC that preceded Zeitlos

E.2 Open Hardware Tools

- **Yosys**, synthesis: https://github.com/YosysHQ/yosys
- **nextpnr**, place and route: https://github.com/YosysHQ/nextpnr
- **Project IceStorm**, Lattice iCE40: https://github.com/YosysHQ/icestorm
- **Project Trellis**, Lattice ECP5: https://github.com/YosysHQ/prjtrellis
- **Project X-Ray**, Xilinx 7-series: https://github.com/f4pga/prjxray
- **Project Peppercorn**, GateMate: https://github.com/YosysHQ/prjpeppercorn
- **Icarus Verilog**, simulation: https://github.com/steveicarus/iverilog
- **PicoRV32**, a RISC-V softcore: https://github.com/YosysHQ/picorv32
- **LiteX**, SOC builder: https://github.com/enjoy-digital/litex

E.3 Standards

- The RISC-V instruction set specifications are published by RISC-V International at https://riscv.org.
- The Revised(4) Report on the Algorithmic Language Scheme (R4RS), 1991, defines the Scheme dialect used in this book. Later reports, such as R5RS and R7RS, are its successors.
- The C language is standardized by ISO; the examples in this book follow the 1999 standard, C99.
- Verilog is standardized as IEEE 1364; the examples follow Verilog-2005.

E.4 Further Reading

These books cover the subjects of this book in more depth. All are widely available in libraries.

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*. How computers work, built up from switches, for any reader.
- Sarah L. Harris and David Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Digital logic, Verilog and processor design, ending with a working RISC-V processor.
- Paul Horowitz and Winfield Hill, *The Art of Electronics*. The standard practical reference on electronics.
- Harold Abelson and Gerald Jay Sussman, with Julie Sussman, *Structure and Interpretation of Computer Programs*. Programming and the nature of programming languages, taught in Scheme.
- Brian Kernighan and Dennis Ritchie, *The C Programming Language*. A concise introduction to C, by Kernighan and C's creator, Ritchie.
- Andrew Tanenbaum and Albert Woodhull, *Operating Systems: Design and Implementation*. Operating systems explained through a complete small microkernel system.
- John Kemeny and Thomas Kurtz, *Back to BASIC*. The creators of BASIC on why they made it and what it was for.